QUANTUM SECURITY INTELLIGENCE

# Security & Post-Quantum Readiness Audit Report

coinbase-cb-mpc-en @ 0b716706f998 · Coinbase — public repository (passive review)

| | |
|---|---|
| Organization | QSI |
| Report date | 2026-11-04 18:42 UTC |
| Report version | 1 |
| Audit run | 32b0f949-8ecd-42bd-b745-74580de74cf7 · Document audit · 2026-11-04 18:42 UTC |
| Analyzer engine | qsi-engine 1.3.0 |
| Risk profile | Crypto exchange, custodian or wallet |
| Content SHA-256 | `f97305d07a10199298791bef31824379c96cf9e73e16151592d84131b0a588d9` |
| Signature key (Ed25519, SHA-256 fingerprint; the signature is the separate file report.sig.json) | `5b3cf9c9bd0fa54447e2b5dfa7a6bab569bd0c666fa46949599e5c348d17083e` |

# Contents

1. Executive summary
2. Scope and boundaries
3. Methodology
4. Assets examined and not examined
5. Architecture overview
6. Overall security posture
7. Findings by severity
8. Detailed findings
9. Evidence
10. Post-quantum assessment
11. Limitations and assumptions
12. Remediation plan
13. Priorities for 30, 60 and 90 days
14. Verification after remediation
15. Tools used
16. Sources and normative documents
17. Appendices

This report presents automated and documented analysis results for the stated scope. It is not a legal opinion, a certification or an attestation of compliance with any standard.

# Executive summary

| 11 | 0 | 0 | 11 | 0 |
|---|---|---|---|---|
| Findings in total | Critical | High | Medium | Low |

| Classification | Count |
|---|---|
| Confirmed technical vulnerability | 0 |
| Known vulnerable component | 0 |
| Confirmed configuration error | 0 |
| Potential cryptographic risk | 0 |
| Long-term post-quantum risk | 11 |
| Improvement recommendation | 0 |
| Insufficient data | 0 |

## Top 5

| ID | Severity | Detailed findings | Priority score |
|---|---|---|---|
| QSI-2026-11868 | Medium | Quantum-vulnerable ECDH: key wrapping and data encryption at rest (5 file(s)) confirmed by the analyst | 43.4 |
| QSI-2026-11863 | Medium | Quantum-vulnerable Paillier: threshold signing and MPC protocols (15 file(s)) confirmed by the analyst | 41.8 |
| QSI-2026-11866 | Medium | Quantum-vulnerable Threshold ECDSA: threshold signing and MPC protocols (13 file(s)) confirmed by the analyst | 41.8 |
| QSI-2026-11867 | Medium | Quantum-vulnerable Ed25519: threshold signing and MPC protocols (9 file(s)) confirmed by the analyst | 41.8 |
| QSI-2026-11869 | Medium | Quantum-vulnerable Pedersen commitment: threshold signing and MPC protocols (4 file(s)) confirmed by the analyst | 41.8 |

Mosca's inequality (x + y > z): x = 10, y = 5, z = 8 → migration would finish too late under this scenario. z is a planning assumption chosen per risk profile, not a prediction. A cryptographically relevant quantum computer is not publicly known to exist today.

Analyst review: 10 confirmed, 0 not applicable, 0 dismissed as false positives.

**Post-quantum maturity: Level 1 of 4 — post-quantum code present, not wired.** Next step: Wire the existing post-quantum primitive into a critical path (key wrapping or the channel) behind a hybrid construction, and test interoperability.

# Scope and boundaries

**Mode:** Document audit

**Repository:** https://github.com/coinbase/cb-mpc @ 0b716706f998633912ee2be9b9bf8909aed66088 — evidence links open the lines at this commit.

## Supplied inputs

174 files, 1453052 bytes in total, 0 redaction(s); the per-file SHA-256 list is in appendix C.

# Methodology

The audit was performed by the QSI engine 1.3.0. Each analyzer emits facts (rule, parameters, evidence); finding texts are generated from the rule catalog. Methodology sources are listed under "Sources and normative documents".

Prioritization uses an explainable weighted model (severity, exploitability, exposure, confidentiality/integrity/availability impact weighted by the "Crypto exchange, custodian or wallet" profile, asset criticality, urgency, ease of remediation) with a confidence multiplier. CVSS v3.1 is applied only to technical vulnerabilities with a defined vector. Long-term quantum risks use Mosca's inequality; their severity is capped at Medium and they are never presented as exploitable vulnerabilities.

Finding classes: confirmed technical vulnerability; known vulnerable component (a dependency version named in an advisory; whether the affected code is reachable is assessed separately, and a component confirmed by review becomes a vulnerability); confirmed configuration error; potential cryptographic risk; long-term post-quantum risk; improvement recommendation; insufficient data. Heuristic findings are marked preliminary; analyst decisions are shown next to the findings.

## Coverage of checks

| Area | Status | Count | Examples |
|---|---|---|---|
| source_code | checked | 174 | include-internal/cbmpc/internal/core/convert.h: 307 lines scanned with 16 rule patterns and 145 registry patterns; include-internal/cbmpc/internal/core/extended_uint.h: 54 lines scanned with 16 rule patterns and 145 registry patterns; include-internal/cbmpc/internal/core/log.h: 124 lines scanned with 16 rule patterns and 145 registry patterns |
| entropy | checked | 1 | random-number generation: 174 code files read with 24 macro definitions from the tree; 0 ineffective guard(s), 0 weak random-bytes provider(s), 0 weak key-generation site(s) |

Long lists are grouped here; the full per-file list is in the JSON report, signed with the same key.

# Assets examined and not examined

**Supplied inputs:** 175 (input files: C)

# Architecture overview

Static review of the source tree coinbase-cb-mpc-en at 0b716706f998633912ee2be9b9bf8909aed66088. 174 files analysed; of 181 files, 0 are test, fuzzing, benchmark or example code and were left out, 7 are not read by any analyzer, 0 exceed the per-file limit and 0 did not fit the total limit. The review was restricted to 4 path(s) of the repository: CMakeLists.txt, include, include-internal, src; 962 files outside them were not read. Files no analyzer reads include: CMakeLists.txt, src/cbmpc/api/CMakeLists.txt, src/cbmpc/c_api/CMakeLists.txt, src/cbmpc/core/CMakeLists.txt, src/cbmpc/crypto/CMakeLists.txt. No requests were sent to the owner's systems.

# Overall security posture

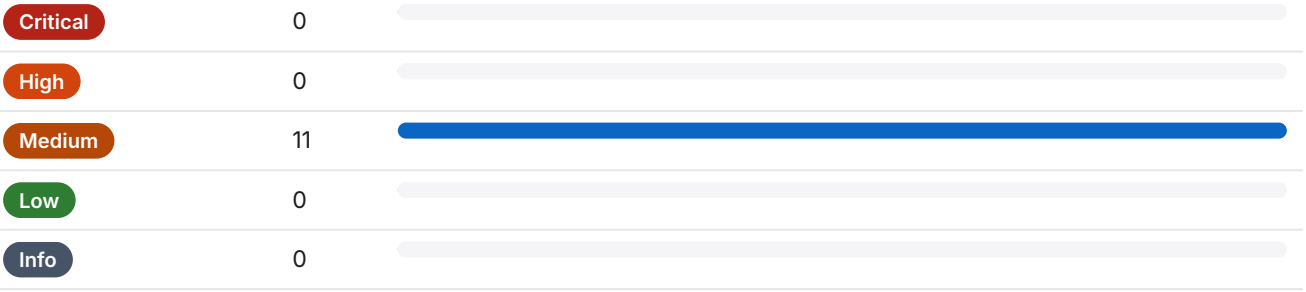
**100**
Indicative posture index (0–100)

**86.1%**
Quantum-vulnerable share of inventory

**108**
Cryptographic inventory

Derived from findings in scope (confirmed and potential issues weighted by severity). Indicative only; not comparable across differently scoped audits.

# Findings by severity




| ID | Detailed findings | Classification | Severity | Confidence |
|---|---|---|---|---|
| QSI-2026-11868 | Quantum-vulnerable ECDH: key wrapping and data encryption at rest (5 file(s)) | Long-term post-quantum risk | Medium | Medium |
| QSI-2026-11863 | Quantum-vulnerable Paillier: threshold signing and MPC protocols (15 file(s)) | Long-term post-quantum risk | Medium | Medium |
| QSI-2026-11866 | Quantum-vulnerable Threshold ECDSA: threshold signing and MPC protocols (13 file(s)) | Long-term post-quantum risk | Medium | Medium |
| QSI-2026-11867 | Quantum-vulnerable Ed25519: threshold signing and MPC protocols (9 file(s)) | Long-term post-quantum risk | Medium | Medium |
| QSI-2026-11869 | Quantum-vulnerable Pedersen commitment: threshold signing and MPC protocols (4 file(s)) | Long-term post-quantum risk | Medium | Medium |
| QSI-2026-11870 | Quantum-vulnerable ECDSA: threshold signing and MPC protocols (3 file(s)) | Long-term post-quantum risk | Medium | Medium |
| QSI-2026-11872 | Quantum-vulnerable ECDSA: transaction and message signing (1 file(s)) | Long-term post-quantum risk | Medium | Medium |
| QSI-2026-11864 | Quantum-vulnerable ECDSA: role not determined (6 file(s)) | Long-term post-quantum risk | Medium | Medium |
| QSI-2026-11871 | Quantum-vulnerable Ed25519: role not determined (3 file(s)) | Long-term post-quantum risk | Medium | Medium |
| QSI-2026-11873 | Quantum-vulnerable RSA: role not determined (1 file(s)) | Long-term post-quantum risk | Medium | Medium |
| QSI-2026-11865 | Quantum-vulnerable Schnorr (BIP-340): role not determined (2 file(s)) | Long-term post-quantum risk | Medium | Medium, preliminary |

# Detailed findings

QSI-2026-11868 · CODE-QV-ASYMMETRIC

Medium

## Quantum-vulnerable ECDH: key wrapping and data encryption at rest (5 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-28): confirmed by the analyst.** The highest-urgency class in this repository, and it is not signing. The anchors are the publicly verifiable encryption paths — include-internal/cbmpc/internal/protocol/pve_base.h:213 calls prv_key.ecdh(...) and checks a 32-byte output, and src/cbmpc/api/{pve_base_pke,pve_batch_ac, pve_batch_single_recipient}.cpp plus pve_internal.h:154 all guard on "missing HSM ECIES ECDH callback". PVE is how key shares are encrypted to a recipient for backup and escrow, so the ciphertexts are stored. Harvest-now-decrypt-later applies in full: a copy taken today of an encrypted share set becomes plaintext once a CRQC exists, and no later migration undoes that. Unlike a signature scheme, this can be fixed today by wrapping the share under a hybrid KEM, which makes it the first thing to do rather than the hardest — and nearer than it looks, because this repository already implements ML-KEM-768 without the PVE path using it. See the DEP-NO-PQC-LIBRARY decision below.

5 reference(s) to ECDH across 5 file(s), e.g. include-internal/cbmpc/internal/protocol/pve_base.h:213. Role, judged from paths, function names and nearby code: key wrapping and data encryption at rest. Keys or data encrypted to these public keys are stored: anyone who copies them now decrypts them once a CRQC exists (harvest now, decrypt later).

| | |
|---|---|
| Affected asset | source-code:ECDH:key_wrapping |
| Data source | include-internal/cbmpc/internal/protocol/pve_base.h |
| Detection method | Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches. |
| Analyzer | source_code 1.3.0 · 2026-11-04 18:42 UTC |
| Priority score | 43.4 / 100 (Crypto exchange, custodian or wallet) |
| CVSS v3.1 | not applicable |
| Mosca's inequality (x + y > z) | x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario |

### POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

### REPRODUCTION STEPS

```
Open include-internal/cbmpc/internal/protocol/pve_base.h:213 and inspect the quoted lines; the evidence lists further files.
```

### EVIDENCE

file_excerpt · include-internal/cbmpc/internal/protocol/pve_base.h · line 213 · SHA-256 02f0b0e2dc3db13d... · File 26dcc28fef1f2733...

```
212 |     if (rv = prv_key.ecdh(prv_key.ctx, prv_key.dk_handle, kem_ct, dh)) return rv;
213 |     if (dh.size() != 32) return coinbase::error(E_CRYPTO, "invalid ECDH output size");
214 |
```

file_excerpt · src/cbmpc/api/pve_base_pke.cpp · line 325 · SHA-256 7a1b1fff636e81f0... · File 5639e943156dbf5e...

```
324 |   if (const error_t rv = coinbase::api::detail::validate_mem_arg(label, "label")) return rv;
325 |   if (!cb.ecdh) return coinbase::error(E_BADARG, "missing HSM ECIES ECDH callback");
326 |
```

file_excerpt · src/cbmpc/api/pve_batch_ac.cpp · line 308 · SHA-256 8e385b276c3dbb90... · File 8b8be49db0d9daba...

```
307 |   if (const error_t rv = coinbase::api::detail::validate_mem_arg(label, "label")) return rv;
308 |   if (!cb.ecdh) return coinbase::error(E_BADARG, "missing HSM ECIES ECDH callback");
309 |
```

file_excerpt · src/cbmpc/api/pve_batch_single_recipient.cpp · line 225 · SHA-256 862c1cebf150e177... · File c5572970684fdeff...

```
224 |   if (const error_t rv = coinbase::api::detail::validate_mem_arg(label, "label")) return rv;
225 |   if (!cb.ecdh) return coinbase::error(E_BADARG, "missing HSM ECIES ECDH callback");
226 |
```

file_excerpt · src/cbmpc/api/pve_internal.h · line 154 · SHA-256 3fa55d0a81efe46a... · File 47bce2585e9a7bf7...

```
153 |   error_t decrypt(mem_t dk_handle, mem_t label, mem_t ct, buf_t& out_plain) const override {
154 |     if (!cb_.ecdh) return coinbase::error(E_BADARG, "missing HSM ECIES ECDH callback");
155 |     coinbase::mpc::pve_ecies_p256_hsm_dk_t dk;
```

### LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

### RECOMMENDATION

Wrap keys with ML-KEM or a hybrid KEM (X25519 + ML-KEM-768) and re-wrap the stored keys; data that may already have been copied needs its own assessment.

### RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-11863 · CODE-QV-ASYMMETRIC

Medium

## Quantum-vulnerable Paillier: threshold signing and MPC protocols (15 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-28): confirmed by the analyst.** Correct, with a parameter worth quoting: include-internal/cbmpc/internal/crypto/base_paillier.h declares class paillier_t with static constexpr int bit_size = 2048, so the modulus is 2048 bits and Shor's algorithm faces exactly the problem it faces for RSA-2048. The evidence is real implementation rather than a name match — base_paillier.h, zk/zk_paillier.h's valid_paillier_t, and the key structures in protocol/ecdsa_2p.h and hd_keyset_ecdsa_2p.h, which each hold a crypto::paillier_t and its c_key. This is the row that a post-quantum *signature* standard does not help with: Paillier is the homomorphic encryption the two-party ECDSA protocol is built on,

> so replacing it is a protocol redesign, and NIST's 2024 standards (ML-DSA, SLH-DSA) are single-signer schemes that do not offer a drop-in replacement.

352 reference(s) to Paillier across 15 file(s), e.g. include-internal/cbmpc/internal/crypto/base_bn.h:46. Role, judged from paths, function names and nearby code: threshold signing and MPC protocols. Paillier, Pedersen commitments and the discrete-logarithm proofs inside threshold ECDSA and EdDSA protocols all fall to Shor's algorithm; a post-quantum threshold scheme is a protocol change, not a parameter change.

| | |
|---|---|
| Affected asset | source-code:Paillier:threshold_mpc |
| Data source | include-internal/cbmpc/internal/crypto/base_bn.h |
| Detection method | Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches. |
| Analyzer | source_code 1.3.0 · 2026-11-04 18:42 UTC |
| Priority score | 41.8 / 100 (Crypto exchange, custodian or wallet) |
| CVSS v3.1 | not applicable |
| Mosca's inequality (x + y > z) | x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario |

### POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

### REPRODUCTION STEPS

```
Open include-internal/cbmpc/internal/crypto/base_bn.h:46 and inspect the quoted lines; the evidence lists further files.
```

### EVIDENCE

file_excerpt · include-internal/cbmpc/internal/crypto/base_bn.h · line 46 · SHA-256 5b3069a225ca12ea… · File 2ab03b1f64160824…

```
45 | class mod_t;
46 | class paillier_t;
47 | class ecdsa_signature_t;
```

file_excerpt · include-internal/cbmpc/internal/crypto/base_bn.h · line 57 · SHA-256 ee1680c40ae2e30e… · File 2ab03b1f64160824…

```
56 |   friend class crypto::ecc_point_t;
57 |   friend class crypto::paillier_t;
58 |   friend class crypto::ecdsa_signature_t;
```

file_excerpt · include-internal/cbmpc/internal/crypto/base_paillier.h · line 8 · SHA-256 5e4a09314f2cd39e… · File 635074e16e9ac9d0…

```
 7 |
 8 | class paillier_t {
 9 |  public:
```

file_excerpt · include-internal/cbmpc/internal/crypto/base_paillier.h · line 29 · SHA-256 5f18c9d760a32724… · File 635074e16e9ac9d0…

```
28 |   static constexpr int bit_size = 2048;
29 |   paillier_t() : has_private(false) {}
30 |   ~paillier_t() {}
```

file_excerpt · include-internal/cbmpc/internal/crypto/base_paillier.h · line 30 · SHA-256 0cb26e46930009ca… · File 635074e16e9ac9d0…

```
29 |   paillier_t() : has_private(false) {}
30 |   ~paillier_t() {}
31 |
```

file_excerpt · include-internal/cbmpc/internal/protocol/ecdsa_2p.h · line 18 · SHA-256 ca851b698b030d55… · File bb2d45c7091bcb2d…

```
17 |   bn_t c_key;
18 |   crypto::paillier_t paillier;
19 | };
```

file_excerpt · include-internal/cbmpc/internal/protocol/ecdsa_2p.h · line 66 · SHA-256 0986d849de59c959… · File bb2d45c7091bcb2d…

```
65 |  */
66 | struct paillier_gen_interactive_t {
67 |   paillier_gen_interactive_t(const crypto::mpc_pid_t& pid) : range(pid), equal(pid) {}
```

file_excerpt · include-internal/cbmpc/internal/protocol/ecdsa_2p.h · line 67 · SHA-256 220d61e7527123b6… · File bb2d45c7091bcb2d…

```
66 | struct paillier_gen_interactive_t {
67 |   paillier_gen_interactive_t(const crypto::mpc_pid_t& pid) : range(pid), equal(pid) {}
68 |
```

file_excerpt · include-internal/cbmpc/internal/protocol/hd_keyset_ecdsa_2p.h · line 11 · SHA-256 6626153477cce1ab… · File 98cef577118dc981…

```
10 |   hd_root_t root;
11 |   coinbase::crypto::paillier_t paillier;
12 |   bn_t c_key;
```

file_excerpt · include-internal/cbmpc/internal/zk/zk_paillier.h · line 9 · SHA-256 b73ec02d5359acd4… · File 5ad37919ff30625b…

```
 8 | struct valid_paillier_t {
 9 |   using param = paillier_non_interactive_param_t;
10 |
```

### LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

### RECOMMENDATION

Track post-quantum threshold signatures (NIST IR 8214C) and keep the protocol layer ready to swap the signature scheme; record which keys and shares must stay valid for how long.

### RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-11866 · CODE-QV-ASYMMETRIC

Medium

## Quantum-vulnerable Threshold ECDSA: threshold signing and MPC protocols (13 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-28): confirmed by the analyst.** The claim is right — cb-mpc is a threshold ECDSA library and these 13 files are its protocol surface (api/ecdsa_2p.h, api/ecdsa_mp.h, internal/protocol/ecdsa_2p.h, ecdsa_mp.h, hd_keyset_ecdsa_2p.h) — but the cited evidence is weaker than the claim and a reader should know which is which. Most anchors are namespace declarations and their closing braces (namespace coinbase::api::ecdsa_2p, coinbase::mpc::ecdsampc), so the row follows from the library's purpose rather than from the quoted line. The substantive evidence for this class is in the Paillier and Pedersen rows above, which cite implementations. Confirmed on that basis, and the detector would serve better by anchoring on the protocol functions.

102 reference(s) to Threshold ECDSA across 13 file(s), e.g. include-internal/cbmpc/internal/protocol/ecdsa_2p.h:10. Role, judged from paths, function names and nearby code: threshold signing and MPC protocols. Paillier, Pedersen commitments and the discrete-logarithm proofs inside threshold ECDSA and EdDSA protocols all fall to Shor's algorithm; a post-quantum threshold scheme is a protocol change, not a parameter change.

| | |
|---|---|
| Affected asset | source-code:Threshold ECDSA:threshold_mpc |
| Data source | include-internal/cbmpc/internal/protocol/ecdsa_2p.h |
| Detection method | Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches. |
| Analyzer | source_code 1.3.0 · 2026-11-04 18:42 UTC |
| Priority score | 41.8 / 100 (Crypto exchange, custodian or wallet) |
| CVSS v3.1 | not applicable |
| Mosca's inequality (x + y > z) | x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario |

### POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

### REPRODUCTION STEPS

```
Open include-internal/cbmpc/internal/protocol/ecdsa_2p.h:10 and inspect the quoted lines; the evidence lists further files.
```

### EVIDENCE

file_excerpt · include-internal/cbmpc/internal/protocol/ecdsa_2p.h · line 10 · SHA-256 4f07cdced92dd607… · File bb2d45c7091bcb2d…

```
 9 |
10 | namespace coinbase::mpc::ecdsa2pc {
11 |
```

file_excerpt · include-internal/cbmpc/internal/protocol/ecdsa_2p.h · line 122 · SHA-256 eb44feaab487c809… · File bb2d45c7091bcb2d…

```
121 |
122 | }  // namespace coinbase::mpc::ecdsa2pc
```

file_excerpt · include-internal/cbmpc/internal/protocol/ecdsa_mp.h · line 9 · SHA-256 867210bf6da87290… · File 09e920ca00569b73…

```
  8 |
  9 | namespace coinbase::mpc::ecdsampc {
 10 |
```

file_excerpt · include-internal/cbmpc/internal/protocol/ecdsa_mp.h · line 129 · SHA-256 1733ee2e1cb11d10… · File 09e920ca00569b73…

```
128 |
129 | }  // namespace coinbase::mpc::ecdsampc
```

file_excerpt · include-internal/cbmpc/internal/protocol/hd_keyset_ecdsa_2p.h · line 3 · SHA-256 219e8141fa597210… · File 98cef577118dc981…

```
  2 | #include <cbmpc/internal/protocol/ec_dkg.h>
  3 | #include <cbmpc/internal/protocol/ecdsa_2p.h>
  4 | #include <cbmpc/internal/protocol/hd_tree_bip32.h>
```

file_excerpt · include-internal/cbmpc/internal/protocol/hd_keyset_ecdsa_2p.h · line 29 · SHA-256 a5b11e730725f117… · File 98cef577118dc981…

```
 28 |                            const std::vector<bip32_path_t>& non_hardened_paths, buf_t&
sid,
 29 |                            std::vector<ecdsa2pc::key_t>& derived_keys);
 30 |
```

file_excerpt · include/cbmpc/api/ecdsa_2p.h · line 12 · SHA-256 a78ce67e456bbade… · File 105375859252d5a6…

```
 11 |
 12 | namespace coinbase::api::ecdsa_2p {
 13 |
```

file_excerpt · include/cbmpc/api/ecdsa_2p.h · line 88 · SHA-256 2a83480fd5d03e50… · File 105375859252d5a6…

```
 87 |
 88 | }  // namespace coinbase::api::ecdsa_2p
```

file_excerpt · include/cbmpc/api/ecdsa_mp.h · line 15 · SHA-256 eb3d5758c2755f9f… · File ab364b34f5c08cdc…

```
 14 | // requirements.
 15 | namespace coinbase::api::ecdsa_mp {
 16 |
```

file_excerpt · include/cbmpc/api/ecdsa_mp.h · line 134 · SHA-256 6496220e2d358496… · File ab364b34f5c08cdc…

```
133 |
134 | }  // namespace coinbase::api::ecdsa_mp
```

## LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

### RECOMMENDATION

Track post-quantum threshold signatures (NIST IR 8214C) and keep the protocol layer ready to swap the signature scheme; record which keys and shares must stay valid for how long.

### RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-11867 · CODE-QV-ASYMMETRIC

Medium

## Quantum-vulnerable Ed25519: threshold signing and MPC protocols (9 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-28): confirmed by the analyst.** Threshold EdDSA, which the library implements alongside threshold ECDSA: api/eddsa_2p.h, internal/protocol/eddsa.h (which includes schnorr_mp.h), hd_keyset_eddsa_2p.h and src/cbmpc/api/eddsa2pc.cpp, where blob_to_key validates the key blob role. Same conclusion as threshold ECDSA and the same caveat about namespace anchors; the protocol is a Schnorr-style multiparty scheme over Ed25519, so Shor breaks both the signature and the discrete-log proofs around it.

38 reference(s) to Ed25519 across 9 file(s), e.g. include-internal/cbmpc/internal/protocol/eddsa.h:6. Role, judged from paths, function names and nearby code: threshold signing and MPC protocols. Paillier, Pedersen commitments and the discrete-logarithm proofs inside threshold ECDSA and EdDSA protocols all fall to Shor's algorithm; a post-quantum threshold scheme is a protocol change, not a parameter change.

| | |
|---|---|
| Affected asset | source-code:Ed25519:threshold_mpc |
| Data source | include-internal/cbmpc/internal/protocol/eddsa.h |
| Detection method | Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches. |
| Analyzer | source_code 1.3.0 · 2026-11-04 18:42 UTC |
| Priority score | 41.8 / 100 (Crypto exchange, custodian or wallet) |
| CVSS v3.1 | not applicable |
| Mosca's inequality (x + y > z) | x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario |

### POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

### REPRODUCTION STEPS

```
Open include-internal/cbmpc/internal/protocol/eddsa.h:6 and inspect the quoted lines; the evidence lists further files.
```

### EVIDENCE

file_excerpt · include-internal/cbmpc/internal/protocol/eddsa.h · line 6 · SHA-256 3c84aa793ed1f429… · File 1bae54da49de50c0…

```
    5 | #include <cbmpc/internal/protocol/schnorr_mp.h>
    6 | namespace coinbase::mpc::eddsa2pc {
    7 |
```

file_excerpt · include-internal/cbmpc/internal/protocol/eddsa.h · line 14 · SHA-256 b312965fb62a137d… · File 1bae54da49de50c0…

```
   13 |
   14 | }  // namespace coinbase::mpc::eddsa2pc
   15 |
```

file_excerpt · include-internal/cbmpc/internal/protocol/hd_keyset_eddsa_2p.h · line 26 · SHA-256 ef05e73154048d30… · File 0ec98767b365b0bc…

```
   25 |                              const std::vector<bip32_path_t>& non_hardened_paths, buf_t&
sid,
   26 |                              std::vector<eddsa2pc::key_t>& derived_keys);
   27 |
```

file_excerpt · include/cbmpc/api/eddsa_2p.h · line 8 · SHA-256 c37e0dee9fd8cf51… · File 16ec1fa4dcb7e197…

```
    7 |
    8 | namespace coinbase::api::eddsa_2p {
    9 |
```

file_excerpt · include/cbmpc/api/eddsa_2p.h · line 79 · SHA-256 1870990aac8e55e0… · File 16ec1fa4dcb7e197…

```
   78 |
   79 | }  // namespace coinbase::api::eddsa_2p
```

file_excerpt · src/cbmpc/api/eddsa2pc.cpp · line 1 · SHA-256 0d8efb61e1de8ae3… · File 277a7491b190d4e1…

```
    1 | #include <cbmpc/api/eddsa_2p.h>
    2 | #include <cbmpc/internal/core/convert.h>
```

file_excerpt · src/cbmpc/api/eddsa2pc.cpp · line 8 · SHA-256 c37e0dee9fd8cf51… · File 277a7491b190d4e1…

```
    7 |
    8 | namespace coinbase::api::eddsa_2p {
    9 |
```

file_excerpt · src/cbmpc/api/eddsa2pc.cpp · line 29 · SHA-256 bebfad4c5352b665… · File 277a7491b190d4e1…

```
   28 |
   29 | static error_t blob_to_key(const key_blob_v1_t& blob, coinbase::mpc::eddsa2pc::key_t& key) {
   30 |   if (blob.role > 1) return coinbase::error(E_FORMAT, "invalid key blob role");
```

file_excerpt · src/cbmpc/api/hd_keyset_eddsa_2p.cpp · line 23 · SHA-256 f5e7d1db58a72a54… · File db69a2795c6ad2b9…

```
   22 | // Mirror of the `coinbase::api::eddsa_2p` key blob format (see `src/cbmpc/api/
eddsa2pc.cpp`).
   23 | constexpr uint32_t eddsa2pc_key_blob_version_v1 = 1;
   24 | struct eddsa2pc_key_blob_v1_t {
```

file_excerpt · src/cbmpc/api/hd_keyset_eddsa_2p.cpp · line 24 · SHA-256 b14eb05d04888ba3… · File db69a2795c6ad2b9…

```
   23 | constexpr uint32_t eddsa2pc_key_blob_version_v1 = 1;
   24 | struct eddsa2pc_key_blob_v1_t {
   25 |   uint32_t version = eddsa2pc_key_blob_version_v1;
```

### LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

### RECOMMENDATION

Track post-quantum threshold signatures (NIST IR 8214C) and keep the protocol layer ready to swap the signature scheme; record which keys and shares must stay valid for how long.

### RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-11869 · CODE-QV-ASYMMETRIC

Medium

## Quantum-vulnerable Pedersen commitment: threshold signing and MPC protocols (4 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-28): confirmed by the analyst.** Correct and anchored on real parameter generation, not a mention: src/cbmpc/zk/zk_pedersen.cpp constructs pedersen_commitment_params_t with primes generated via BN_generate_prime_ex (the comment names the OpenSSL function), zk_pedersen.h:13 exposes get() and check_safe_prime_subgroup, and both src/cbmpc/protocol/ecdsa_2p.cpp:28 and src/cbmpc/zk/zk_paillier.cpp:537 pull those parameters. Stated precisely, because the distinction decides how bad this is: breaking the discrete log costs a Pedersen commitment its *binding* property, not its hiding — the commitment is perfectly hiding, so no committed value is recoverable from it even with a CRQC. The exposure is forgeability inside the proofs, i.e. future protocol soundness, not retroactive disclosure.

21 reference(s) to Pedersen commitment across 4 file(s), e.g. include-internal/cbmpc/internal/zk/zk_pedersen.h:8. Role, judged from paths, function names and nearby code: threshold signing and MPC protocols. Paillier, Pedersen commitments and the discrete-logarithm proofs inside threshold ECDSA and EdDSA protocols all fall to Shor's algorithm; a post-quantum threshold scheme is a protocol change, not a parameter change.

| | |
|---|---|
| Affected asset | source-code:Pedersen commitment:threshold_mpc |
| Data source | include-internal/cbmpc/internal/zk/zk_pedersen.h |
| Detection method | Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches. |
| Analyzer | source_code 1.3.0 · 2026-11-04 18:42 UTC |
| Priority score | 41.8 / 100 (Crypto exchange, custodian or wallet) |
| CVSS v3.1 | not applicable |
| Mosca's inequality (x + y > z) | x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario |

### POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

### REPRODUCTION STEPS

```
Open include-internal/cbmpc/internal/zk/zk_pedersen.h:8 and inspect the quoted lines; the evidence
lists further files.
```

### EVIDENCE

file_excerpt · include-internal/cbmpc/internal/zk/zk_pedersen.h · line 8 · SHA-256 90f6fa4e7aa38c51... · File 899fad68ffd0cc05...

```
  7 |
  8 | struct pedersen_commitment_params_t {
  9 |   mod_t p, p_tag;
```

file_excerpt · include-internal/cbmpc/internal/zk/zk_pedersen.h · line 13 · SHA-256 beb6763be217e297... · File 899fad68ffd0cc05...

```
 12 |
 13 |   static const pedersen_commitment_params_t& get();
 14 |   error_t check_safe_prime_subgroup(const bn_t& x) const;
```

file_excerpt · include-internal/cbmpc/internal/zk/zk_pedersen.h · line 17 · SHA-256 4a5620026d361a84... · File 899fad68ffd0cc05...

```
 16 |  private:
 17 |   pedersen_commitment_params_t();
 18 | };
```

file_excerpt · src/cbmpc/protocol/ecdsa_2p.cpp · line 28 · SHA-256 f6cd7432a9262739... · File 676da782f28e8371...

```
 27 |
 28 |   const zk::pedersen_commitment_params_t& params = zk::pedersen_commitment_params_t::get();
 29 |   const mod_t& p_tag = params.p_tag;
```

file_excerpt · src/cbmpc/zk/zk_paillier.cpp · line 537 · SHA-256 cebe585f3f001af3... · File fd1dc137a7611808...

```
536 |                                     const bn_t& r, mem_t session_id, uint64_t aux) {
537 |   const pedersen_commitment_params_t& params = pedersen_commitment_params_t::get();
538 |   const mod_t& p_tag = params.p_tag;
```

file_excerpt · src/cbmpc/zk/zk_pedersen.cpp · line 6 · SHA-256 edeeb6e35af2f2df... · File 968641640c623fc2...

```
  5 |
  6 | pedersen_commitment_params_t::pedersen_commitment_params_t() {
  7 |   // this was generated using https://www.openssl.org/docs/man1.1.1/man3/
BN_generate_prime_ex.html
```

file_excerpt · src/cbmpc/zk/zk_pedersen.cpp · line 39 · SHA-256 59b6c7a5b07fdfda... · File 968641640c623fc2...

```
 38 |
 39 | const pedersen_commitment_params_t& pedersen_commitment_params_t::get() {
 40 |   static pedersen_commitment_params_t params;
```

file_excerpt · src/cbmpc/zk/zk_pedersen.cpp · line 40 · SHA-256 ad78a5b9a922f859... · File 968641640c623fc2...

```
 39 | const pedersen_commitment_params_t& pedersen_commitment_params_t::get() {
 40 |   static pedersen_commitment_params_t params;
 41 |   return params;
```

### LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

### RECOMMENDATION

Track post-quantum threshold signatures (NIST IR 8214C) and keep the protocol layer ready to swap the signature scheme; record which keys and shares must stay valid for how long.

### RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-11870 · CODE-QV-ASYMMETRIC

Medium

## Quantum-vulnerable ECDSA: threshold signing and MPC protocols (3 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-28): confirmed by the analyst.** Plain (non-threshold) ECDSA on secp256k1 inside the Schnorr two-party API: src/cbmpc/api/schnorr2pc.cpp:31 and :48 validate and write blob.curve against curve_id::secp256k1 and serialise key.Q compressed. Key-blob handling for the signing keys, so the same migration item as the threshold rows; counted separately because it is the serialisation format, which is where a migration needs a version bump.

17 reference(s) to ECDSA across 3 file(s), e.g. src/cbmpc/api/schnorr2pc.cpp:31. Role, judged from paths, function names and nearby code: threshold signing and MPC protocols. Paillier, Pedersen commitments and the discrete-logarithm proofs inside threshold ECDSA and EdDSA protocols all fall to Shor's algorithm; a post-quantum threshold scheme is a protocol change, not a parameter change.

| | |
|---|---|
| Affected asset | source-code:ECDSA:threshold_mpc |
| Data source | src/cbmpc/api/schnorr2pc.cpp |
| Detection method | Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches. |
| Analyzer | source_code 1.3.0 · 2026-11-04 18:42 UTC |
| Priority score | 41.8 / 100 (Crypto exchange, custodian or wallet) |
| CVSS v3.1 | not applicable |
| Mosca's inequality (x + y > z) | x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario |

### POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

### REPRODUCTION STEPS

```
Open src/cbmpc/api/schnorr2pc.cpp:31 and inspect the quoted lines; the evidence lists further files.
```

### EVIDENCE

file_excerpt · src/cbmpc/api/schnorr2pc.cpp · line 31 · SHA-256 781d30e36a4e8dc2... · File 9495eab05f3110ea...

```
   30 |   if (blob.role > 1) return coinbase::error(E_FORMAT, "invalid key blob role");
   31 |   if (static_cast<curve_id>(blob.curve) != curve_id::secp256k1)
   32 |     return coinbase::error(E_FORMAT, "invalid key blob curve");
```

file_excerpt · src/cbmpc/api/schnorr2pc.cpp · line 48 · SHA-256 70c0040ad2b59e8f... · File 9495eab05f3110ea...

```
   47 |   blob.role = static_cast<uint32_t>(key.role);
   48 |   blob.curve = static_cast<uint32_t>(curve_id::secp256k1);
   49 |   blob.Q_compressed = key.Q.to_compressed_bin();
```

file_excerpt · src/cbmpc/api/schnorr2pc.cpp · line 67 · SHA-256 8e0b65f6eab20ce3... · File 9495eab05f3110ea...

```
   66 |   if (const error_t rv = validate_job_2p(job)) return rv;
   67 |   if (curve != curve_id::secp256k1) return coinbase::error(E_BADARG, "unsupported curve");
   68 |
```

file_excerpt · src/cbmpc/api/schnorr_mp.cpp · line 46 · SHA-256 6b5c5538004c224d... · File bd909c04140c085c...

```
   45 |     return coinbase::error(E_FORMAT, "unsupported key blob version");
   46 |   if (static_cast<curve_id>(blob.curve) != curve_id::secp256k1)
   47 |     return coinbase::error(E_FORMAT, "invalid key blob curve");
```

file_excerpt · src/cbmpc/api/schnorr_mp.cpp · line 71 · SHA-256 6082a2958ef6b914... · File bd909c04140c085c...

```
   70 |   blob.version = version;
   71 |   blob.curve = static_cast<uint32_t>(curve_id::secp256k1);
   72 |   blob.party_name = key.party_name;
```

file_excerpt · src/cbmpc/api/schnorr_mp.cpp · line 117 · SHA-256 516efb1988607679... · File bd909c04140c085c...

```
  116 |     return coinbase::error(E_FORMAT, "unsupported key blob version: " +
std::to_string(blob.version));
  117 |   if (static_cast<curve_id>(blob.curve) != curve_id::secp256k1)
  118 |     return coinbase::error(E_FORMAT, "invalid key blob curve");
```

file_excerpt · src/cbmpc/protocol/schnorr_2p.cpp · line 82 · SHA-256 6933ccbb7668499b... · File a998fa463bf10815...

```
   81 |   if (variant == variant_e::BIP340) {
   82 |     if (curve != crypto::curve_secp256k1) return coinbase::error(E_BADARG, "BIP340 variant
requires secp256k1 curve");
   83 |     for (int i = 0; i < n_sigs; i++) {
```

file_excerpt · src/cbmpc/protocol/schnorr_2p.cpp · line 136 · SHA-256 a382d78dd9af5355... · File a998fa463bf10815...

```
  135 |         if (rv = crypto::bip340::verify(pub_key, msgs[i], candidate_sigs[i]))
  136 |           return coinbase::error(rv, "schnorr_2p: secp256k1 verify failed");
  137 |       } else {
```

### LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability.

Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

### RECOMMENDATION

Track post-quantum threshold signatures (NIST IR 8214C) and keep the protocol layer ready to swap the signature scheme; record which keys and shares must stay valid for how long.

### RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-11872 · CODE-QV-ASYMMETRIC

Medium

## Quantum-vulnerable ECDSA: transaction and message signing (1 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-28): confirmed by the analyst.** Plain (non-threshold) ECDSA on secp256k1 inside the Schnorr two-party API: src/cbmpc/api/schnorr2pc.cpp:31 and :48 validate and write blob.curve against curve_id::secp256k1 and serialise key.Q compressed. Key-blob handling for the signing keys, so the same migration item as the threshold rows; counted separately because it is the serialisation format, which is where a migration needs a version bump.

45 reference(s) to ECDSA across 1 file(s), e.g. src/cbmpc/crypto/base_ecc_secp256k1.cpp:5. Role, judged from paths, function names and nearby code: transaction and message signing. A CRQC derives the private key from a public key that is already known (on chain after the first spend, in reused addresses): funds and approvals controlled by these keys need a migration path.

| | |
|---|---|
| Affected asset | source-code:ECDSA:transaction_signing |
| Data source | src/cbmpc/crypto/base_ecc_secp256k1.cpp |
| Detection method | Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches. |
| Analyzer | source_code 1.3.0 · 2026-11-04 18:42 UTC |
| Priority score | 41.8 / 100 (Crypto exchange, custodian or wallet) |
| CVSS v3.1 | not applicable |
| Mosca's inequality (x + y > z) | x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario |

### POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

### REPRODUCTION STEPS

```
Open src/cbmpc/crypto/base_ecc_secp256k1.cpp:5 and inspect the quoted lines; the evidence lists further files.
```

### EVIDENCE

file_excerpt · src/cbmpc/crypto/base_ecc_secp256k1.cpp · line 5 · SHA-256 872667182b513b2f… · File 31cabe4ca8532731…

```
4 | // clang-format off
5 | #include "secp256k1/src/assumptions.h"
6 | #include "secp256k1/src/field_impl.h"
```

file_excerpt · src/cbmpc/crypto/base_ecc_secp256k1.cpp · line 6 · SHA-256 080665ef1f83af11… · File 31cabe4ca8532731…

```
5 | #include "secp256k1/src/assumptions.h"
6 | #include "secp256k1/src/field_impl.h"
7 | #include "secp256k1/src/scalar_impl.h"
```

file_excerpt · src/cbmpc/crypto/base_ecc_secp256k1.cpp · line 7 · SHA-256 3fef17948caa87b9… · File 31cabe4ca8532731…

```
6 | #include "secp256k1/src/field_impl.h"
7 | #include "secp256k1/src/scalar_impl.h"
8 | #include "secp256k1/src/scratch_impl.h"
```

### LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

### RECOMMENDATION

Follow each chain's post-quantum roadmap, avoid public-key reuse, and keep signers ready to add post-quantum signature schemes as chains adopt them.

### RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-11864 · CODE-QV-ASYMMETRIC

Medium

## Quantum-vulnerable ECDSA: role not determined (6 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-28): confirmed by the analyst.** Plain (non-threshold) ECDSA on secp256k1 inside the Schnorr two-party API: src/cbmpc/api/schnorr2pc.cpp:31 and :48 validate and write blob.curve against curve_id::secp256k1 and serialise key.Q compressed. Key-blob handling for the signing keys, so the same migration item as the threshold rows; counted separately because it is the serialisation format, which is where a migration needs a version bump.

12 reference(s) to ECDSA across 6 file(s), e.g. include-internal/cbmpc/internal/crypto/base_ecc.h:17. Role, judged from paths, function names and nearby code: role not determined. The code does not show what the keys protect; review decides the priority.

| | |
|---|---|
| Affected asset | source-code:ECDSA:general |
| Data source | include-internal/cbmpc/internal/crypto/base_ecc.h |
| Detection method | Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches. |

| | |
|---|---|
| Analyzer | source_code 1.3.0 · 2026-11-04 18:42 UTC |
| Priority score | 40.2 / 100 (Crypto exchange, custodian or wallet) |
| CVSS v3.1 | not applicable |
| Mosca's inequality (x + y > z) | x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario |

### POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

### REPRODUCTION STEPS

```
Open include-internal/cbmpc/internal/crypto/base_ecc.h:17 and inspect the quoted lines; the evidence lists further files.
```

### EVIDENCE

file_excerpt · include-internal/cbmpc/internal/crypto/base_ecc.h · line 17 · SHA-256 1acb65be89f9287e... · File 4622c8393c4f4bf8...

```
  16 |
  17 | namespace secp256k1 {
  18 | typedef struct point_t* point_ptr_t;
```

file_excerpt · include-internal/cbmpc/internal/crypto/base_ecc.h · line 177 · SHA-256 32e38a7fe938b9f8... · File 4622c8393c4f4bf8...

```
 176 |   explicit ecc_point_t(ecurve_t curve, const ecp_storage_t& storage);
 177 |   explicit ecc_point_t(const secp256k1::point_ptr_t p);
 178 |
```

file_excerpt · include-internal/cbmpc/internal/crypto/base_ecc.h · line 246 · SHA-256 d6d38a0587aa30e2... · File 4622c8393c4f4bf8...

```
 245 |     EC_POINT* ptr = nullptr;
 246 |     secp256k1::point_ptr_t secp256k1;
 247 |     ecp_storage_t* storage;
```

file_excerpt · include/cbmpc/api/curve.h · line 9 · SHA-256 8bc94e0040057c98... · File 2facde4582730feb...

```
   8 | enum class curve_id : uint32_t {
   9 |   p256 = 1,       // NIST P-256 (aka prime256v1 / secp256r1)
  10 |   secp256k1 = 2,  // secp256k1
```

file_excerpt · include/cbmpc/api/curve.h · line 10 · SHA-256 0eb2996e7229d0e7... · File 2facde4582730feb...

```
   9 |   p256 = 1,       // NIST P-256 (aka prime256v1 / secp256r1)
  10 |   secp256k1 = 2,  // secp256k1
  11 |   ed25519 = 3,    // Edwards25519
```

file_excerpt · src/cbmpc/api/curve_util.h · line 13 · SHA-256 4a75918a0824abc5... · File 2b1ce8031d294bd7...

```
  12 |       return coinbase::crypto::curve_p256;
  13 |     case curve_id::secp256k1:
  14 |       return coinbase::crypto::curve_secp256k1;
```

file_excerpt · src/cbmpc/api/curve_util.h · line 30 · SHA-256 ea0e22518060778a... · File 2b1ce8031d294bd7...

```
  29 |   if (curve == coinbase::crypto::curve_secp256k1) {
  30 |     out = curve_id::secp256k1;
  31 |     return true;
```

file_excerpt · src/cbmpc/c_api/util.h · line 79 · SHA-256 e7c477a077d0efbf... · File 54e197b005d5bf85...

```
 78 |     case CBMPC_CURVE_SECP256K1:
 79 |       out = coinbase::api::curve_id::secp256k1;
 80 |       return CBMPC_SUCCESS;
```

file_excerpt · src/cbmpc/crypto/base_ecc.cpp · line 673 · SHA-256 59055bc4efcd046e... · File dd31645addc8cf00...

```
672 |
673 | namespace secp256k1 {
674 | point_ptr_t new_point(const point_ptr_t);
```

file_excerpt · src/cbmpc/crypto/base_ecc.cpp · line 677 · SHA-256 8996735a795dea2a... · File dd31645addc8cf00...

```
676 |
677 | ecc_point_t::ecc_point_t(const secp256k1::point_ptr_t p)
678 |     : curve(curve_secp256k1)  // NOLINT:cppcoreguidelines-pro-type-member-init
```

### LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

### RECOMMENDATION

Classify each usage, record data lifetimes, and plan migration to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it; introduce an abstraction layer for crypto-agility.

### RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-11871 · CODE-QV-ASYMMETRIC

Medium

## Quantum-vulnerable Ed25519: role not determined (3 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-28): confirmed by the analyst.** Threshold EdDSA, which the library implements alongside threshold ECDSA: api/eddsa_2p.h, internal/protocol/eddsa.h (which includes schnorr_mp.h), hd_keyset_eddsa_2p.h and src/cbmpc/api/eddsa2pc.cpp, where blob_to_key validates the key blob role. Same conclusion as threshold ECDSA and the same caveat about namespace anchors; the protocol is a Schnorr-style multiparty scheme over Ed25519, so Shor breaks both the signature and the discrete-log proofs around it.

27 reference(s) to Ed25519 across 3 file(s), e.g. src/cbmpc/crypto/base_ecc.cpp:420. Role, judged from paths, function names and nearby code: role not determined. The code does not show what the keys protect; review decides the priority.

| | |
|---|---|
| Affected asset | source-code:Ed25519:general |
| Data source | src/cbmpc/crypto/base_ecc.cpp |

| | |
|---|---|
| Detection method | Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches. |
| Analyzer | source_code 1.3.0 · 2026-11-04 18:42 UTC |
| Priority score | 40.2 / 100 (Crypto exchange, custodian or wallet) |
| CVSS v3.1 | not applicable |
| Mosca's inequality (x + y > z) | x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario |

## POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

## REPRODUCTION STEPS

```
Open src/cbmpc/crypto/base_ecc.cpp:420 and inspect the quoted lines; the evidence lists further
files.
```

## EVIDENCE

file_excerpt · src/cbmpc/crypto/base_ecc.cpp · line 420 · SHA-256 bc97f1d43e02c893… · File dd31645addc8cf00…

```
  419 | void ecc_prv_key_t::set_ed_bin(mem_t ed_bin) {
  420 |   cb_assert(ed_bin.size == ed25519::prv_bin_size());
  421 |   this->curve = curve_ed25519;
```

file_excerpt · src/cbmpc/crypto/base_ecc.cpp · line 428 · SHA-256 7704d598e6d557af… · File dd31645addc8cf00…

```
  427 |   bn_t x = val;
  428 |   if (!ed_bin.empty()) x = ed25519::prv_key_to_scalar(ed_bin);
  429 |   return x % curve.order();
```

file_excerpt · src/cbmpc/crypto/base_eddsa.cpp · line 73 · SHA-256 098cb191d292ec1d… · File 45697966a1eb28d1…

```
   72 | void ecurve_ed_t::get_params(bn_t& p, bn_t& a, bn_t& b) const {
   73 |   a = ed25519::a();
   74 |   b = ed25519::b();
```

file_excerpt · src/cbmpc/crypto/base_eddsa.cpp · line 74 · SHA-256 6ac622113d6a0eee… · File 45697966a1eb28d1…

```
   73 |   a = ed25519::a();
   74 |   b = ed25519::b();
   75 |   p = ed25519::p();
```

file_excerpt · src/cbmpc/crypto/base_eddsa.cpp · line 75 · SHA-256 5456bbfafd04245e… · File 45697966a1eb28d1…

```
   74 |   b = ed25519::b();
   75 |   p = ed25519::p();
   76 | }
```

file_excerpt · src/cbmpc/protocol/schnorr_mp.cpp · line 169 · SHA-256 dbde3ddb4baf2c21… · File b7aa492e64186b64…

```
  168 |       for (size_t l = 0; l < msgs.size(); l++) {
  169 |         candidate_sigs[l] = R[l].to_compressed_bin() +
ss[l].to_bin(crypto::ed25519::prv_bin_size()).rev();
  170 |         if (rv = verify_key.verify(msgs[l], candidate_sigs[l])) return coinbase::error(rv,
"ed25519 verify failed");
```

## LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability.

Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

### RECOMMENDATION

Classify each usage, record data lifetimes, and plan migration to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it; introduce an abstraction layer for crypto-agility.

### RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-11873 · CODE-QV-ASYMMETRIC

Medium

## Quantum-vulnerable RSA: role not determined (1 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-28): confirmed by the analyst.** Role resolved. src/cbmpc/crypto/base_rsa.cpp implements rsa_pub_key_t with encrypt_raw (deliberately RSA_NO_PADDING, because it is a primitive for a higher-level scheme rather than a general-purpose encryption API), verify_pkcs1, and EVP_RSA_gen(bits) for generation. It is included by internal/crypto/base_pki.h and internal/crypto/base_paillier.h and exercised by tests/unit/api/test_pve.cpp, so this is the PKI layer underneath publicly verifiable encryption and the modulus arithmetic that Paillier is built on. Same quantum exposure as the Paillier row, for the same reason: Shor factors the composite modulus. Because PVE ciphertexts are stored, this inherits the harvest-now-decrypt-later urgency of the ECDH row rather than the lower urgency of a signing path.

2 reference(s) to RSA across 1 file(s), e.g. src/cbmpc/crypto/base_rsa.cpp:96. Role, judged from paths, function names and nearby code: role not determined. The code does not show what the keys protect; review decides the priority.

| | |
|---|---|
| Affected asset | source-code:RSA:general |
| Data source | src/cbmpc/crypto/base_rsa.cpp |
| Detection method | Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches. |
| Analyzer | source_code 1.3.0 · 2026-11-04 18:42 UTC |
| Priority score | 40.2 / 100 (Crypto exchange, custodian or wallet) |
| CVSS v3.1 | not applicable |
| Mosca's inequality (x + y > z) | x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario |

### POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

### REPRODUCTION STEPS

```
Open src/cbmpc/crypto/base_rsa.cpp:96 and inspect the quoted lines; the evidence lists further
files.
```

### EVIDENCE

file_excerpt · src/cbmpc/crypto/base_rsa.cpp · line 96 · SHA-256 db6acbfbb01b1b6b... · File ce1f7e2af4ab105d...

```
  95 |
  96 |   if (EVP_PKEY_id(pkey) != EVP_PKEY_RSA) {
  97 |     // Handle error: Not an RSA key
```

file_excerpt · src/cbmpc/crypto/base_rsa.cpp · line 201 · SHA-256 2a07b530bd9526bf... · File ce1f7e2af4ab105d...

```
 200 |   create();
 201 |   ptr = EVP_RSA_gen(bits);
 202 |   cb_assert(ptr);
```

### LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

### RECOMMENDATION

Classify each usage, record data lifetimes, and plan migration to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it; introduce an abstraction layer for crypto-agility.

### RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-11865 · CODE-QV-ASYMMETRIC

Medium

## Quantum-vulnerable Schnorr (BIP-340): role not determined (2 file(s))

Long-term post-quantum risk | Confidence: Medium | preliminary — requires further verification

4 reference(s) to Schnorr (BIP-340) across 2 file(s), e.g. include-internal/cbmpc/internal/crypto/base_ecc.h:299. Role, judged from paths, function names and nearby code: role not determined. The code does not show what the keys protect; review decides the priority.

| | |
|---|---|
| Affected asset | source-code:Schnorr (BIP-340):general |
| Data source | include-internal/cbmpc/internal/crypto/base_ecc.h |
| Detection method | Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches. |
| Analyzer | source_code 1.3.0 · 2026-11-04 18:42 UTC |
| Priority score | 34.2 / 100 (Crypto exchange, custodian or wallet) |
| CVSS v3.1 | not applicable |
| Mosca's inequality (x + y > z) | x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario |

### POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

### REPRODUCTION STEPS

```
Open include-internal/cbmpc/internal/crypto/base_ecc.h:299 and inspect the quoted lines; the
evidence lists further files.
```

### EVIDENCE

file_excerpt · include-internal/cbmpc/internal/crypto/base_ecc.h · line 299 · SHA-256 28f82276a8490953… · File 4622c8393c4f4bf8…

```
298 |
299 |   error_t verify_schnorr(mem_t message, mem_t signature) const;
300 | };
```

file_excerpt · include-internal/cbmpc/internal/crypto/base_ecc.h · line 328 · SHA-256 7743eba94a1a6606… · File 4622c8393c4f4bf8…

```
327 |
328 |   buf_t sign_schnorr(mem_t hash) const;
329 |   sig_with_pub_key_t sign_and_output_pub_key(mem_t hash) const;
```

file_excerpt · src/cbmpc/crypto/base_ecc.cpp · line 373 · SHA-256 a6d9f37f314044dd… · File dd31645addc8cf00…

```
372 |
373 | error_t ecc_pub_key_t::verify_schnorr(mem_t message, mem_t signature) const {
374 |   if (signature.size != curve.size() * 2) return coinbase::error(E_FORMAT);
```

file_excerpt · src/cbmpc/crypto/base_ecc.cpp · line 485 · SHA-256 890e2d1fab1145ca… · File dd31645addc8cf00…

```
484 |
485 | buf_t ecc_prv_key_t::sign_schnorr(mem_t message) const {
486 |   const auto& G = curve.generator();
```

### LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

### RECOMMENDATION

Classify each usage, record data lifetimes, and plan migration to a post-quantum algorithm; introduce an abstraction layer for crypto-agility.

### RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

# Evidence

| ID | Type | Data source | SHA-256 |
|---|---|---|---|
| QSI-2026-11868 | file_excerpt | include-internal/cbmpc/internal/protocol/pve_base.h (line 213) | 02f0b0e2dc3db13d04880ba70b571232395b33294953c92a27a9d1 |
| QSI-2026-11868 | file_excerpt | src/cbmpc/api/pve_base_pke.cpp (line 325) | 7a1b1fff636e81f0627ac69da87fc93e1fcf0bf121c4e2297b0718 |
| QSI-2026-11868 | file_excerpt | src/cbmpc/api/pve_batch_ac.cpp (line 308) | 8e385b276c3dbb900f226e3840ad803a3463b08dc1a1f91ea26225 |
| QSI-2026-11868 | file_excerpt | src/cbmpc/api/pve_batch_single_recipient.cpp (line 225) | 862c1cebf150e17798084f5554fc57ca003f1addc2df7e38232744 |
| QSI-2026-11868 | file_excerpt | src/cbmpc/api/pve_internal.h (line 154) | 3fa55d0a81efe46a0c8448fd5d5943231913648274934d50546cae |
| QSI-2026-11863 | file_excerpt | include-internal/cbmpc/internal/crypto/base_bn.h (line 46) | 5b3069a225ca12ea3e28ab0aeabfda51b549f1b3b9d656edc897fe |
| QSI-2026-11863 | file_excerpt | include-internal/cbmpc/internal/crypto/base_bn.h (line 57) | ee1680c40ae2e30e376737e08f755915785323d13d53983ed66744 |
| QSI-2026-11863 | file_excerpt | include-internal/cbmpc/internal/crypto/base_paillier.h (line 8) | 5e4a09314f2cd39ed8e8283c855838bd6ca755c5868d316db91eee |
| QSI-2026-11863 | file_excerpt | include-internal/cbmpc/internal/crypto/base_paillier.h (line 29) | 5f18c9d760a32724c050c5db7a67fbc655776bda0da0b20a58aad0 |
| QSI-2026-11863 | file_excerpt | include-internal/cbmpc/internal/crypto/base_paillier.h (line 30) | 0cb26e46930009ca686d8c0d8cf27ded9d756b290f72260c02b479 |
| QSI-2026-11863 | file_excerpt | include-internal/cbmpc/internal/protocol/ecdsa_2p.h (line 18) | ca851b698b030d5580b83a036c3c776223f359db508a167eef1d50 |
| QSI-2026-11863 | file_excerpt | include-internal/cbmpc/internal/protocol/ecdsa_2p.h (line 66) | 0986d849de59c959f92b00dd88d412729fbe9ff0da5f142cd1c102 |
| QSI-2026-11863 | file_excerpt | include-internal/cbmpc/internal/protocol/ecdsa_2p.h (line 67) | 220d61e7527123b6ac429da5f9c81c381cf521d6a0dd872c9c236e |
| QSI-2026-11863 | file_excerpt | include-internal/cbmpc/internal/protocol/hd_keyset_ecdsa_2p.h (line 11) | 6626153477cce1abe743ecfb348e91e140577229a70e4d4dc32b35 |
| QSI-2026-11863 | file_excerpt | include-internal/cbmpc/internal/zk/zk_paillier.h (line 9) | b73ec02d5359acd4ea08e3bf49c6d23136c6a5d7bc3ff5e36548fa |

| | | | |
|---|---|---|---|
| QSI-2026-11866 | file_excerpt | include-internal/cbmpc/internal/protocol/ecdsa_2p.h (line 10) | `4f07cdced92dd607c8b26f039382244c79cc890b96ad3e83064608` |
| QSI-2026-11866 | file_excerpt | include-internal/cbmpc/internal/protocol/ecdsa_2p.h (line 122) | `eb44feaab487c80962c7a431cfbdf3361a9a37156029f2ec3d6219` |
| QSI-2026-11866 | file_excerpt | include-internal/cbmpc/internal/protocol/ecdsa_mp.h (line 9) | `867210bf6da87290b3d8ca0486277ad9893eb8ba3d71cb1b0f72a3` |
| QSI-2026-11866 | file_excerpt | include-internal/cbmpc/internal/protocol/ecdsa_mp.h (line 129) | `1733ee2e1cb11d10a703f1fc869b155110678b46904534af78db5d` |
| QSI-2026-11866 | file_excerpt | include-internal/cbmpc/internal/protocol/hd_keyset_ecdsa_2p.h (line 3) | `219e8141fa597210bbafd00b034030aa8b243102ba810d0f7e1e7b` |
| QSI-2026-11866 | file_excerpt | include-internal/cbmpc/internal/protocol/hd_keyset_ecdsa_2p.h (line 29) | `a5b11e730725f1172506a636b59dc27320599bd992208ea7d2320c` |
| QSI-2026-11866 | file_excerpt | include/cbmpc/api/ecdsa_2p.h (line 12) | `a78ce67e456bbade8e40693fbf9637650f04faa2e8891063ddeebe` |
| QSI-2026-11866 | file_excerpt | include/cbmpc/api/ecdsa_2p.h (line 88) | `2a83480fd5d03e50bebb35352c4cb14708fc498793961d1a8abf5d` |
| QSI-2026-11866 | file_excerpt | include/cbmpc/api/ecdsa_mp.h (line 15) | `eb3d5758c2755f9f5a315a84c4a64733cdf65c78f998b14de387f2` |
| QSI-2026-11866 | file_excerpt | include/cbmpc/api/ecdsa_mp.h (line 134) | `6496220e2d3584960750a9402b32c82c32f580a8ea5e20a24637f0` |
| QSI-2026-11867 | file_excerpt | include-internal/cbmpc/internal/protocol/eddsa.h (line 6) | `3c84aa793ed1f42936617a69b2658887b6d94795e53413cf8e1516` |
| QSI-2026-11867 | file_excerpt | include-internal/cbmpc/internal/protocol/eddsa.h (line 14) | `b312965fb62a137d3aae3bf51910e3d5803f05ece5bcf33078f902` |
| QSI-2026-11867 | file_excerpt | include-internal/cbmpc/internal/protocol/hd_keyset_eddsa_2p.h (line 26) | `ef05e73154048d3043c22b41a17397906767a18a0b8cf741d1cf19` |
| QSI-2026-11867 | file_excerpt | include/cbmpc/api/eddsa_2p.h (line 8) | `c37e0dee9fd8cf516ded4812f075b46479e94f71599bc145a46dd6` |
| QSI-2026-11867 | file_excerpt | include/cbmpc/api/eddsa_2p.h (line 79) | `1870990aac8e55e075d1ff1db55dfa554b8413e8081ccb3390a7b8` |
| QSI-2026-11867 | file_excerpt | src/cbmpc/api/eddsa2pc.cpp (line 1) | `0d8efb61e1de8ae34931c96eb484fc2f9c4ff617d0efa53fce450b` |

| | | | |
|---|---|---|---|
| QSI-2026-11867 | file_excerpt | src/cbmpc/api/eddsa2pc.cpp (line 8) | c37e0dee9fd8cf516ded4812f075b46479e94f71599bc145a46dd6 |
| QSI-2026-11867 | file_excerpt | src/cbmpc/api/eddsa2pc.cpp (line 29) | bebfad4c5352b665f0c5800512a735abd85c681f783086cf92dba6 |
| QSI-2026-11867 | file_excerpt | src/cbmpc/api/hd_keyset_eddsa_2p.cpp (line 23) | f5e7d1db58a72a542c853835f4f649afc616439e30444f917e9a53 |
| QSI-2026-11867 | file_excerpt | src/cbmpc/api/hd_keyset_eddsa_2p.cpp (line 24) | b14eb05d04888ba3a76cb7c9a40d1ecaabd069bdadb57f2e0c8ec3 |
| QSI-2026-11869 | file_excerpt | include-internal/cbmpc/internal/zk/zk_pedersen.h (line 8) | 90f6fa4e7aa38c51851aec3f52cc5042b5a7d6cb58864e285ff279 |
| QSI-2026-11869 | file_excerpt | include-internal/cbmpc/internal/zk/zk_pedersen.h (line 13) | beb6763be217e297856056988084ecbeb400ac2d7bf25b188e8104 |
| QSI-2026-11869 | file_excerpt | include-internal/cbmpc/internal/zk/zk_pedersen.h (line 17) | 4a5620026d361a84899501efab6d448f6aaca1ec3470c5f81cf655 |
| QSI-2026-11869 | file_excerpt | src/cbmpc/protocol/ecdsa_2p.cpp (line 28) | f6cd7432a9262739e96d7886c92f3c7506f35897ea1c58e6393e83 |
| QSI-2026-11869 | file_excerpt | src/cbmpc/zk/zk_paillier.cpp (line 537) | cebe585f3f001af319bb63895f689e137cb8b6b68d327245e8efe2 |
| QSI-2026-11869 | file_excerpt | src/cbmpc/zk/zk_pedersen.cpp (line 6) | edeeb6e35af2f2dfeb59c3d504b58d0c9676130e51cab7743e716f |
| QSI-2026-11869 | file_excerpt | src/cbmpc/zk/zk_pedersen.cpp (line 39) | 59b6c7a5b07fdfda0bfbf3c3cf9123cee14bec4790d944ee6da5b7 |
| QSI-2026-11869 | file_excerpt | src/cbmpc/zk/zk_pedersen.cpp (line 40) | ad78a5b9a922f859724ed7a18381e655d3c93f4063474ab5e7f836 |
| QSI-2026-11870 | file_excerpt | src/cbmpc/api/schnorr2pc.cpp (line 31) | 781d30e36a4e8dc2b8788c3413940215330fa9c99e7ede1cd51753 |
| QSI-2026-11870 | file_excerpt | src/cbmpc/api/schnorr2pc.cpp (line 48) | 70c0040ad2b59e8fc028bfe2b304bf1e5c190dcd9bd81c45122946 |
| QSI-2026-11870 | file_excerpt | src/cbmpc/api/schnorr2pc.cpp (line 67) | 8e0b65f6eab20ce3e7beabdf092159341fdbb8bce48e0965e11375 |
| QSI-2026-11870 | file_excerpt | src/cbmpc/api/schnorr_mp.cpp (line 46) | 6b5c5538004c224df21888f2ab7eb3b2283655671ec668e89d3b44 |
| QSI-2026-11870 | file_excerpt | src/cbmpc/api/schnorr_mp.cpp (line 71) | 6082a2958ef6b9148a404e06a13378b5ddeabf80c823bc9a75ba85 |
| QSI-2026-11870 | file_excerpt | src/cbmpc/api/schnorr_mp.cpp (line 117) | 516efb19886076797c71fe18d00e39cff6d00e5b16854aee71e290 |
| QSI-2026-11870 | file_excerpt | src/cbmpc/protocol/schnorr_2p.cpp (line 82) | 6933ccbb7668499bfc93936f9461308c76da160293ca96d96dc726 |

| | | | |
|---|---|---|---|
| QSI-2026-11870 | file_excerpt | src/cbmpc/protocol/schnorr_2p.cpp (line 136) | a382d78dd9af5355999427605a0c64b9d3262c18a6290782f8046e |
| QSI-2026-11872 | file_excerpt | src/cbmpc/crypto/base_ecc_secp256k1.cpp (line 5) | 872667182b513b2fb3be5898f64aee3035d69bc5401c5422a09fa4 |
| QSI-2026-11872 | file_excerpt | src/cbmpc/crypto/base_ecc_secp256k1.cpp (line 6) | 080665ef1f83af114ec0d02cf611f582583d388fba9861259f38e4 |
| QSI-2026-11872 | file_excerpt | src/cbmpc/crypto/base_ecc_secp256k1.cpp (line 7) | 3fef17948caa87b9d5b3b241d1f94b1459b96d31c44dc1188c4062 |
| QSI-2026-11864 | file_excerpt | include-internal/cbmpc/internal/crypto/base_ecc.h (line 17) | 1acb65be89f9287ebf0f9fdd748084d46a04587d02e8e23bd77cb5 |
| QSI-2026-11864 | file_excerpt | include-internal/cbmpc/internal/crypto/base_ecc.h (line 177) | 32e38a7fe938b9f819247a3f2b8f4009a857ae01cfb18dc6643580 |
| QSI-2026-11864 | file_excerpt | include-internal/cbmpc/internal/crypto/base_ecc.h (line 246) | d6d38a0587aa30e2f8e219bbeb332a262f5d6e28c8f16c55803bbb |
| QSI-2026-11864 | file_excerpt | include/cbmpc/api/curve.h (line 9) | 8bc94e0040057c98486af0d2761abfd6ed943aa9547cb545354de9 |
| QSI-2026-11864 | file_excerpt | include/cbmpc/api/curve.h (line 10) | 0eb2996e7229d0e76b10f0da6cc1353d0ed4e76d9997664d08fa47 |
| QSI-2026-11864 | file_excerpt | src/cbmpc/api/curve_util.h (line 13) | 4a75918a0824abc51d4df2f3b0053de2ad333c16fee4e66dc47eac |
| QSI-2026-11864 | file_excerpt | src/cbmpc/api/curve_util.h (line 30) | ea0e22518060778a7a9df31ae979e7d72520b1fc3bebb5996fceb1 |
| QSI-2026-11864 | file_excerpt | src/cbmpc/c_api/util.h (line 79) | e7c477a077d0efbf921de86527e4f38cd37dec90e2fabbe063db58 |
| QSI-2026-11864 | file_excerpt | src/cbmpc/crypto/base_ecc.cpp (line 673) | 59055bc4efcd046ef6d49e3274cfc060c68a3620ea7436b2d76a3d |
| QSI-2026-11864 | file_excerpt | src/cbmpc/crypto/base_ecc.cpp (line 677) | 8996735a795dea2a0988f9d89bf72c598a082466689a5c52c12992 |
| QSI-2026-11871 | file_excerpt | src/cbmpc/crypto/base_ecc.cpp (line 420) | bc97f1d43e02c8937d050b6ff40221cbbe0b44782536f3749da34f |
| QSI-2026-11871 | file_excerpt | src/cbmpc/crypto/base_ecc.cpp (line 428) | 7704d598e6d557afa1663d4d9e33459c337911a17c2ca8640e59bc |
| QSI-2026-11871 | file_excerpt | src/cbmpc/crypto/base_eddsa.cpp (line 73) | 098cb191d292ec1d0e482d879f64faec8854c3290ae06e8b92fb31 |
| QSI-2026-11871 | file_excerpt | src/cbmpc/crypto/base_eddsa.cpp (line 74) | 6ac622113d6a0eee7bdfb0992c54cd1dfdeabf37880dd29b486199 |
| QSI-2026-11871 | file_excerpt | src/cbmpc/crypto/base_eddsa.cpp (line 75) | 5456bbfafd04245e9627dce77292b49b426863b9ecd3f6de7dffa7 |

| | | | |
|---|---|---|---|
| QSI-2026-11871 | file_excerpt | src/cbmpc/protocol/schnorr_mp.cpp (line 169) | `dbde3ddb4baf2c21baf4a87a6914ed7f9e0289e5a9e6080e665bb0` |
| QSI-2026-11873 | file_excerpt | src/cbmpc/crypto/base_rsa.cpp (line 96) | `db6acbfbb01b1b6b2b382dc3e96fd9dbe38bd0d17244c79529f41f` |
| QSI-2026-11873 | file_excerpt | src/cbmpc/crypto/base_rsa.cpp (line 201) | `2a07b530bd9526bfbff8696f00367319e11578e9fbdb917f5d310f` |
| QSI-2026-11865 | file_excerpt | include-internal/cbmpc/internal/crypto/base_ecc.h (line 299) | `28f82276a84909538a7777f2fddddb21be288af66f48428f791704` |
| QSI-2026-11865 | file_excerpt | include-internal/cbmpc/internal/crypto/base_ecc.h (line 328) | `7743eba94a1a6606ace6bf98cf07e363ed6b5af001f037832fd5d2` |
| QSI-2026-11865 | file_excerpt | src/cbmpc/crypto/base_ecc.cpp (line 373) | `a6d9f37f314044dd61e5a40ad96ef3ed821bef576af77319a04b13` |
| QSI-2026-11865 | file_excerpt | src/cbmpc/crypto/base_ecc.cpp (line 485) | `890e2d1fab1145cadf730e5f70917a0ef8d98c0ed2c921e05febab` |

# Post-quantum assessment

| | Key establishment (x = how long data must stay confidential; harvest now, decrypt later) | Signatures (x = how long signatures/ keys must stay trusted) |
|---|---|---|
| Data/key lifetime x | 10 | 10 |
| Migration time y | 5 | 5 |
| Assumed years to CRQC z | 8 (early) | 8 |
| x + y − z | 7 → migration would finish too late under this scenario | 7 → migration would finish too late under this scenario |

Planning assumptions: x for key establishment, x for signatures, y are the defaults of the risk profile “Crypto exchange, custodian or wallet”, not the client's figures; with the client's actual lifetimes the verdict can change.

## Post-quantum maturity: Level 1 of 4 — post-quantum code present, not wired

- Present, not wired: ML-KEM — include-internal/cbmpc/internal/crypto/mlkem768.h
- Present, not wired: ML-KEM — src/cbmpc/crypto/mlkem768.cpp
- Classical only — key wrapping and data encryption at rest: ECDH (5 file(s))
- Classical only — threshold signing and MPC protocols: ECDSA, Ed25519, Paillier, Pedersen commitment, Threshold ECDSA (33 file(s))

**Next step:** Wire the existing post-quantum primitive into a critical path (key wrapping or the channel) behind a hybrid construction, and test interoperability.

Levels: 0 none; 1 present, not wired; 2 in use outside the critical paths; 3 hybrid in a critical path; 4 post-quantum by default in the critical paths (root of trust, key wrapping, channel key exchange). Judged from code and manifests.

## What the quantum-vulnerable algorithms do

| Role | Algorithms (files) | ID | First step |
|---|---|---|---|
| **key wrapping and data encryption at rest** | ECDH (5) | QSI-2026-11868 | Wrap keys with ML-KEM or a hybrid KEM (X25519 + ML-KEM-768) and re-wrap the stored keys; data that may already have been copied needs its own assessment |
| **threshold signing and MPC protocols** | Paillier (15), Threshold ECDSA (13), Ed25519 (9), Pedersen commitment (4), ECDSA (3) | QSI-2026-11863, QSI-2026-11866, QSI-2026-11867, QSI-2026-11869, QSI-2026-11870 | Track post-quantum threshold signatures (NIST IR 8214C) and keep the protocol layer ready to swap the signature scheme; record which keys and shares must stay valid for how long |
| **transaction and message signing** | ECDSA (1) | QSI-2026-11872 | Follow each chain's post-quantum roadmap, avoid public-key reuse, and keep signers ready to add post-quantum signature schemes as chains adopt them |

| | | | |
|---|---|---|---|
| **role not determined** | ECDSA (6), Ed25519 (3), Schnorr (BIP-340) (2), RSA (1) | QSI-2026-11864, QSI-2026-11865, QSI-2026-11871, QSI-2026-11873 | Classify each usage, record data lifetimes, and plan migration to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it; introduce an abstraction layer for crypto-agility |

Roles are judged from file paths, function names and the code around each match; review confirms them. Root-of-trust keys and wrapped keys come first: they are hard to replace or can be recorded today.

z is a planning assumption chosen per risk profile, not a prediction. A cryptographically relevant quantum computer is not publicly known to exist today.

| Quantum status | Count |
|---|---|
| weak classically | 2 |
| reduced margin (Grover), approved | 6 |
| Grover: acceptable to NIST, below ECCG/ANSSI guidance | 5 |
| post-quantum | 2 |
| vulnerable (Shor) | 93 |

**Detailed findings:** QSI-2026-11868, QSI-2026-11863, QSI-2026-11866, QSI-2026-11867, QSI-2026-11869, QSI-2026-11870, QSI-2026-11872, QSI-2026-11864, QSI-2026-11871, QSI-2026-11873, QSI-2026-11865

## PQC readiness questionnaire

No readiness questionnaire was completed for this project.

## Quantum algorithms relevant to this inventory

| Quantum algorithm | Effect | Inventoried algorithms (rows) | What it means for this audit |
|---|---|---|---|
| Shor's algorithm — factoring<br>Q1: quantum computation offline, classical access to the victim | breaks | Paillier (15), RSA (3) | Breaks RSA at every key length: a longer modulus only raises the cost polynomially. Plan migration to ML-KEM (or a hybrid) for key establishment and ML-DSA / SLH-DSA for signatures; start with keys protecting data that must stay confidential for years (harvest now, decrypt later) and with long-lived signatures. |

| | | | |
|---|---|---|---|
| Shor's algorithm — discrete logarithms in finite fields<br>Q1: quantum computation offline, classical access to the victim | breaks | BN254 (alt_bn128) (2), ElGamal (4) | Breaks finite-field Diffie–Hellman and DSA at every size. Move key establishment to ML-KEM (hybrid with X25519 or ECDH during the transition) and signatures to ML-DSA or SLH-DSA. |
| Shor's algorithm — elliptic-curve discrete logarithms<br>Q1: quantum computation offline, classical access to the victim | breaks | BN254 (alt_bn128) (2), ECDH (5), ECDSA (26), Ed25519 (19), Pedersen commitment (4), Schnorr (BIP-340) (2), Threshold ECDSA (13) | Elliptic-curve keys are expected to fall before RSA keys of comparable classical strength. On blockchains, coins whose public key is already on chain (reused addresses, pay-to-public-key and Taproot outputs) are exposed first; plan the move to post-quantum signatures and avoid key reuse in the meantime. |
| Grover's algorithm — key search and preimages<br>Q1: quantum computation offline, classical access to the victim | weakens (larger keys restore the margin) | AES-128 (1), AES-192 (1), AES-256 (1), BLAKE2 (1), HKDF (3), HMAC (1), RIPEMD-160 (1), SHA-1 (1), SHA-2 (2), SHA-3 (1) | Not a break. 256-bit keys (AES-256, ChaCha20) keep a comfortable margin; AES-128 meets NIST category 1, while some authorities ask for 256-bit keys in quantum-resistant contexts (see the AES-128 entry). Prefer 256-bit keys for long-lived data; 256-bit hashes need no change for preimage resistance. |
| Brassard–Høyer–Tapp — collision search<br>Q1: quantum computation offline, classical access to the victim | weakens (larger keys restore the margin) | BLAKE2 (1), RIPEMD-160 (1), SHA-1 (1), SHA-2 (2), SHA-3 (1) | 256-bit hashes (SHA-256, SHA3-256) remain adequate for collision resistance; 160-bit hashes are already weak classically (SHA-1 collisions are practical). Use 384/512-bit hashes where NIST categories 4–5 are required. |
| Quantum lattice sieving<br>Q1: quantum computation offline, classical access to the victim | included in standard parameters | ML-KEM (2) | No change for ML-KEM or ML-DSA at the standard parameter sets. A 2024 claim of a polynomial-time quantum LWE algorithm was withdrawn after a bug was found (see claims). |

Effects: breaks — replace the algorithm; weakens — larger keys or outputs restore the margin; included in standard parameters — no action at standard parameter sets; only with superposition queries — not applicable to ordinary deployments.

# Limitations and assumptions

- This report presents automated and documented analysis results for the stated scope. It is not a legal opinion, a certification or an attestation of compliance with any standard.
- z is a planning assumption chosen per risk profile, not a prediction. A cryptographically relevant quantum computer is not publicly known to exist today.

# Remediation plan

QSI-2026-11868 · P3 · 90 days

short-term risk reduction

### Add ECDH usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

| | |
|---|---|
| Rationale | Migration cannot be planned without knowing where and why the algorithm is used. |
| Expected outcome | Every usage has an owner, a lifetime and a migration path. |
| Complexity | medium |
| Dependencies | Access to system owners. |
| Side effects | None (documentation and design work). |
| Tests before change | Confirm the inventory entry matches the deployed configuration. |
| Tests after change | Inventory review signed off by the system owner. |
| Closure criteria | Inventory entry with owner, lifetime and target algorithm exists. |

QSI-2026-11863 · P3 · 90 days

short-term risk reduction

### Add Paillier usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

| | |
|---|---|
| Rationale | Migration cannot be planned without knowing where and why the algorithm is used. |
| Expected outcome | Every usage has an owner, a lifetime and a migration path. |
| Complexity | medium |
| Dependencies | Access to system owners. |
| Side effects | None (documentation and design work). |
| Tests before change | Confirm the inventory entry matches the deployed configuration. |
| Tests after change | Inventory review signed off by the system owner. |
| Closure criteria | Inventory entry with owner, lifetime and target algorithm exists. |

QSI-2026-11866 · P3 · 90 days

short-term risk reduction

### Add Threshold ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

| | |
|---|---|
| Rationale | Migration cannot be planned without knowing where and why the algorithm is used. |
| Expected outcome | Every usage has an owner, a lifetime and a migration path. |
| Complexity | medium |
| Dependencies | Access to system owners. |
| Side effects | None (documentation and design work). |
| Tests before change | Confirm the inventory entry matches the deployed configuration. |
| Tests after change | Inventory review signed off by the system owner. |

| | |
|---|---|
| Closure criteria | Inventory entry with owner, lifetime and target algorithm exists. |

QSI-2026-11867 · P3 · 90 days

short-term risk reduction

### Add Ed25519 usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

| | |
|---|---|
| Rationale | Migration cannot be planned without knowing where and why the algorithm is used. |
| Expected outcome | Every usage has an owner, a lifetime and a migration path. |
| Complexity | medium |
| Dependencies | Access to system owners. |
| Side effects | None (documentation and design work). |
| Tests before change | Confirm the inventory entry matches the deployed configuration. |
| Tests after change | Inventory review signed off by the system owner. |
| Closure criteria | Inventory entry with owner, lifetime and target algorithm exists. |

QSI-2026-11869 · P3 · 90 days

short-term risk reduction

### Add Pedersen commitment usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

| | |
|---|---|
| Rationale | Migration cannot be planned without knowing where and why the algorithm is used. |
| Expected outcome | Every usage has an owner, a lifetime and a migration path. |
| Complexity | medium |
| Dependencies | Access to system owners. |
| Side effects | None (documentation and design work). |
| Tests before change | Confirm the inventory entry matches the deployed configuration. |
| Tests after change | Inventory review signed off by the system owner. |
| Closure criteria | Inventory entry with owner, lifetime and target algorithm exists. |

QSI-2026-11870 · P3 · 90 days

short-term risk reduction

### Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

| | |
|---|---|
| Rationale | Migration cannot be planned without knowing where and why the algorithm is used. |
| Expected outcome | Every usage has an owner, a lifetime and a migration path. |
| Complexity | medium |
| Dependencies | Access to system owners. |
| Side effects | None (documentation and design work). |
| Tests before change | Confirm the inventory entry matches the deployed configuration. |
| Tests after change | Inventory review signed off by the system owner. |

| | |
|---|---|
| Closure criteria | Inventory entry with owner, lifetime and target algorithm exists. |

QSI-2026-11872 · P3 · 90 days

short-term risk reduction

### Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

| | |
|---|---|
| Rationale | Migration cannot be planned without knowing where and why the algorithm is used. |
| Expected outcome | Every usage has an owner, a lifetime and a migration path. |
| Complexity | medium |
| Dependencies | Access to system owners. |
| Side effects | None (documentation and design work). |
| Tests before change | Confirm the inventory entry matches the deployed configuration. |
| Tests after change | Inventory review signed off by the system owner. |
| Closure criteria | Inventory entry with owner, lifetime and target algorithm exists. |

QSI-2026-11864 · P3 · 90 days

short-term risk reduction

### Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

| | |
|---|---|
| Rationale | Migration cannot be planned without knowing where and why the algorithm is used. |
| Expected outcome | Every usage has an owner, a lifetime and a migration path. |
| Complexity | medium |
| Dependencies | Access to system owners. |
| Side effects | None (documentation and design work). |
| Tests before change | Confirm the inventory entry matches the deployed configuration. |
| Tests after change | Inventory review signed off by the system owner. |
| Closure criteria | Inventory entry with owner, lifetime and target algorithm exists. |

QSI-2026-11871 · P3 · 90 days

short-term risk reduction

### Add Ed25519 usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

| | |
|---|---|
| Rationale | Migration cannot be planned without knowing where and why the algorithm is used. |
| Expected outcome | Every usage has an owner, a lifetime and a migration path. |
| Complexity | medium |
| Dependencies | Access to system owners. |
| Side effects | None (documentation and design work). |
| Tests before change | Confirm the inventory entry matches the deployed configuration. |
| Tests after change | Inventory review signed off by the system owner. |

| | |
|---|---|
| Closure criteria | Inventory entry with owner, lifetime and target algorithm exists. |

QSI-2026-11873 · P3 · 90 days

short-term risk reduction

### Add RSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

| | |
|---|---|
| Rationale | Migration cannot be planned without knowing where and why the algorithm is used. |
| Expected outcome | Every usage has an owner, a lifetime and a migration path. |
| Complexity | medium |
| Dependencies | Access to system owners. |
| Side effects | None (documentation and design work). |
| Tests before change | Confirm the inventory entry matches the deployed configuration. |
| Tests after change | Inventory review signed off by the system owner. |
| Closure criteria | Inventory entry with owner, lifetime and target algorithm exists. |

QSI-2026-11865 · P3 · 90 days

short-term risk reduction

### Add Schnorr (BIP-340) usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

| | |
|---|---|
| Rationale | Migration cannot be planned without knowing where and why the algorithm is used. |
| Expected outcome | Every usage has an owner, a lifetime and a migration path. |
| Complexity | medium |
| Dependencies | Access to system owners. |
| Side effects | None (documentation and design work). |
| Tests before change | Confirm the inventory entry matches the deployed configuration. |
| Tests after change | Inventory review signed off by the system owner. |
| Closure criteria | Inventory entry with owner, lifetime and target algorithm exists. |

QSI-2026-11868 · P4 · 90 days

root-cause fix

### Plan migration of ECDH to Hybrid key establishment with ML-KEM (FIPS 203), e.g. X25519MLKEM768 in TLS 1.3 or mlkem768x25519-sha256 in SSH (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

| | |
|---|---|
| Rationale | Removes dependence on a quantum-vulnerable public-key primitive. |
| Expected outcome | The asset uses a NIST-standardized post-quantum or hybrid algorithm. |
| Complexity | high |
| Dependencies | Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers. |
| Side effects | Larger keys/signatures, possible performance and compatibility impact. |
| Tests before change | Interoperability tests in a staging environment. |
| Tests after change | Verify the negotiated/used algorithm on the production asset. |
| Closure criteria | |

| | |
|---|---|
| | Re-audit shows Hybrid key establishment with ML-KEM (FIPS 203), e.g. X25519MLKEM768 in TLS 1.3 or mlkem768x25519-sha256 in SSH or a hybrid scheme in use. |

QSI-2026-11863 · P4 · 90 days

root-cause fix

### Plan migration of Paillier to No standardized post-quantum replacement. Plan threshold schemes over post-quantum signatures as they mature (NIST threshold call, IR 8214C), or protocols that avoid Paillier (oblivious-transfer based) (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

| | |
|---|---|
| Rationale | Removes dependence on a quantum-vulnerable public-key primitive. |
| Expected outcome | The asset uses a NIST-standardized post-quantum or hybrid algorithm. |
| Complexity | high |
| Dependencies | Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers. |
| Side effects | Larger keys/signatures, possible performance and compatibility impact. |
| Tests before change | Interoperability tests in a staging environment. |
| Tests after change | Verify the negotiated/used algorithm on the production asset. |
| Closure criteria | Re-audit shows No standardized post-quantum replacement. Plan threshold schemes over post-quantum signatures as they mature (NIST threshold call, IR 8214C), or protocols that avoid Paillier (oblivious-transfer based) or a hybrid scheme in use. |

QSI-2026-11866 · P4 · 90 days

root-cause fix

### Plan migration of Threshold ECDSA to The signature scheme is fixed by the blockchain or verifier. Track chain-level post-quantum proposals and threshold post-quantum signatures (NIST IR 8214C); keep key-share refresh and crypto-agility in the custody design (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

| | |
|---|---|
| Rationale | Removes dependence on a quantum-vulnerable public-key primitive. |
| Expected outcome | The asset uses a NIST-standardized post-quantum or hybrid algorithm. |
| Complexity | high |
| Dependencies | Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers. |
| Side effects | Larger keys/signatures, possible performance and compatibility impact. |
| Tests before change | Interoperability tests in a staging environment. |
| Tests after change | Verify the negotiated/used algorithm on the production asset. |
| Closure criteria | Re-audit shows The signature scheme is fixed by the blockchain or verifier. Track chain-level post-quantum proposals and threshold post-quantum signatures (NIST IR 8214C); keep key-share refresh and crypto-agility in the custody design or a hybrid scheme in use. |

QSI-2026-11867 · P4 · 90 days

root-cause fix

### Plan migration of Ed25519 to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

| | |
|---|---|
| Rationale | Removes dependence on a quantum-vulnerable public-key primitive. |
| Expected outcome | The asset uses a NIST-standardized post-quantum or hybrid algorithm. |
| Complexity | high |
| Dependencies | Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers. |
| Side effects | Larger keys/signatures, possible performance and compatibility impact. |
| Tests before change | Interoperability tests in a staging environment. |
| Tests after change | Verify the negotiated/used algorithm on the production asset. |
| Closure criteria | Re-audit shows ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it or a hybrid scheme in use. |

QSI-2026-11869 · P4 · 90 days

root-cause fix

### Plan migration of Pedersen commitment to (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

| | |
|---|---|
| Rationale | Removes dependence on a quantum-vulnerable public-key primitive. |
| Expected outcome | The asset uses a NIST-standardized post-quantum or hybrid algorithm. |
| Complexity | high |
| Dependencies | Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers. |
| Side effects | Larger keys/signatures, possible performance and compatibility impact. |
| Tests before change | Interoperability tests in a staging environment. |
| Tests after change | Verify the negotiated/used algorithm on the production asset. |
| Closure criteria | Re-audit shows or a hybrid scheme in use. |

QSI-2026-11870 · P4 · 90 days

root-cause fix

### Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

| | |
|---|---|
| Rationale | Removes dependence on a quantum-vulnerable public-key primitive. |
| Expected outcome | The asset uses a NIST-standardized post-quantum or hybrid algorithm. |
| Complexity | high |
| Dependencies | Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers. |
| Side effects | Larger keys/signatures, possible performance and compatibility impact. |
| Tests before change | Interoperability tests in a staging environment. |
| Tests after change | Verify the negotiated/used algorithm on the production asset. |
| Closure criteria | Re-audit shows ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it or a hybrid scheme in use. |

QSI-2026-11872 · P4 · 90 days root-cause fix

### Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

| | |
|---|---|
| Rationale | Removes dependence on a quantum-vulnerable public-key primitive. |
| Expected outcome | The asset uses a NIST-standardized post-quantum or hybrid algorithm. |
| Complexity | high |
| Dependencies | Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers. |
| Side effects | Larger keys/signatures, possible performance and compatibility impact. |
| Tests before change | Interoperability tests in a staging environment. |
| Tests after change | Verify the negotiated/used algorithm on the production asset. |
| Closure criteria | Re-audit shows ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it or a hybrid scheme in use. |

QSI-2026-11864 · P4 · 90 days root-cause fix

### Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

| | |
|---|---|
| Rationale | Removes dependence on a quantum-vulnerable public-key primitive. |
| Expected outcome | The asset uses a NIST-standardized post-quantum or hybrid algorithm. |
| Complexity | high |
| Dependencies | Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers. |
| Side effects | Larger keys/signatures, possible performance and compatibility impact. |
| Tests before change | Interoperability tests in a staging environment. |
| Tests after change | Verify the negotiated/used algorithm on the production asset. |
| Closure criteria | Re-audit shows ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it or a hybrid scheme in use. |

QSI-2026-11871 · P4 · 90 days root-cause fix

### Plan migration of Ed25519 to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

| | |
|---|---|
| Rationale | Removes dependence on a quantum-vulnerable public-key primitive. |
| Expected outcome | The asset uses a NIST-standardized post-quantum or hybrid algorithm. |
| Complexity | high |
| Dependencies | Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers. |
| Side effects | Larger keys/signatures, possible performance and compatibility impact. |
| Tests before change | Interoperability tests in a staging environment. |
| Tests after change | Verify the negotiated/used algorithm on the production asset. |
| Closure criteria | |

| | |
|---|---|
| | Re-audit shows ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it or a hybrid scheme in use. |

QSI-2026-11873 · P4 · 90 days

root-cause fix

### Plan migration of RSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

| | |
|---|---|
| Rationale | Removes dependence on a quantum-vulnerable public-key primitive. |
| Expected outcome | The asset uses a NIST-standardized post-quantum or hybrid algorithm. |
| Complexity | high |
| Dependencies | Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers. |
| Side effects | Larger keys/signatures, possible performance and compatibility impact. |
| Tests before change | Interoperability tests in a staging environment. |
| Tests after change | Verify the negotiated/used algorithm on the production asset. |
| Closure criteria | Re-audit shows ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it or a hybrid scheme in use. |

QSI-2026-11865 · P4 · 90 days

root-cause fix

### Plan migration of Schnorr (BIP-340) to (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

| | |
|---|---|
| Rationale | Removes dependence on a quantum-vulnerable public-key primitive. |
| Expected outcome | The asset uses a NIST-standardized post-quantum or hybrid algorithm. |
| Complexity | high |
| Dependencies | Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers. |
| Side effects | Larger keys/signatures, possible performance and compatibility impact. |
| Tests before change | Interoperability tests in a staging environment. |
| Tests after change | Verify the negotiated/used algorithm on the production asset. |
| Closure criteria | Re-audit shows or a hybrid scheme in use. |

# Priorities for 30, 60 and 90 days

## 30 days

| Priority | ID | Action | Type |
|---|---|---|---|
| None | | | |

## 60 days

| Priority | ID | Action | Type |
|---|---|---|---|
| None | | | |

## 90 days

| Priority | ID | Action | Type |
|---|---|---|---|
| P3 | QSI-2026-11868 | Add ECDH usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers). | short-term risk reduction |
| P3 | QSI-2026-11863 | Add Paillier usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers). | short-term risk reduction |
| P3 | QSI-2026-11866 | Add Threshold ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers). | short-term risk reduction |
| P3 | QSI-2026-11867 | Add Ed25519 usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers). | short-term risk reduction |
| P3 | QSI-2026-11869 | Add Pedersen commitment usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers). | short-term risk reduction |
| P3 | QSI-2026-11870 | Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers). | short-term risk reduction |
| P3 | QSI-2026-11872 | Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers). | short-term risk reduction |
| P3 | QSI-2026-11864 | Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers). | short-term risk reduction |
| P3 | QSI-2026-11871 | Add Ed25519 usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers). | short-term risk reduction |
| P3 | QSI-2026-11873 | Add RSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers). | short-term risk reduction |

| | | | |
|---|---|---|---|
| P3 | QSI-2026-11865 | Add Schnorr (BIP-340) usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers). | short-term risk reduction |
| P4 | QSI-2026-11868 | Plan migration of ECDH to Hybrid key establishment with ML-KEM (FIPS 203), e.g. X25519MLKEM768 in TLS 1.3 or mlkem768x25519-sha256 in SSH (or a hybrid scheme) once supported by your libraries, protocols and counterparties. | root-cause fix |
| P4 | QSI-2026-11863 | Plan migration of Paillier to No standardized post-quantum replacement. Plan threshold schemes over post-quantum signatures as they mature (NIST threshold call, IR 8214C), or protocols that avoid Paillier (oblivious-transfer based) (or a hybrid scheme) once supported by your libraries, protocols and counterparties. | root-cause fix |
| P4 | QSI-2026-11866 | Plan migration of Threshold ECDSA to The signature scheme is fixed by the blockchain or verifier. Track chain-level post-quantum proposals and threshold post-quantum signatures (NIST IR 8214C); keep key-share refresh and crypto-agility in the custody design (or a hybrid scheme) once supported by your libraries, protocols and counterparties. | root-cause fix |
| P4 | QSI-2026-11867 | Plan migration of Ed25519 to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties. | root-cause fix |
| P4 | QSI-2026-11869 | Plan migration of Pedersen commitment to (or a hybrid scheme) once supported by your libraries, protocols and counterparties. | root-cause fix |
| P4 | QSI-2026-11870 | Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties. | root-cause fix |
| P4 | QSI-2026-11872 | Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties. | root-cause fix |
| P4 | QSI-2026-11864 | Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties. | root-cause fix |
| P4 | QSI-2026-11871 | Plan migration of Ed25519 to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties. | root-cause fix |
| P4 | QSI-2026-11873 | Plan migration of RSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties. | root-cause fix |
| P4 | QSI-2026-11865 | Plan migration of Schnorr (BIP-340) to (or a hybrid scheme) once supported by your libraries, protocols and counterparties. | root-cause fix |

# Verification after remediation

| ID | Retest procedure |
|---|---|
| QSI-2026-11868 | Re-run the inventory after refactoring. |
| QSI-2026-11863 | Re-run the inventory after refactoring. |
| QSI-2026-11866 | Re-run the inventory after refactoring. |
| QSI-2026-11867 | Re-run the inventory after refactoring. |
| QSI-2026-11869 | Re-run the inventory after refactoring. |
| QSI-2026-11870 | Re-run the inventory after refactoring. |
| QSI-2026-11872 | Re-run the inventory after refactoring. |
| QSI-2026-11864 | Re-run the inventory after refactoring. |
| QSI-2026-11871 | Re-run the inventory after refactoring. |
| QSI-2026-11873 | Re-run the inventory after refactoring. |
| QSI-2026-11865 | Re-run the inventory after refactoring. |

# Tools used

| Tool | Version |
|---|---|
| blockchain | 1.0.0 |
| certificates | 1.2.0 |
| chain_map | 1.0.0 |
| configs | 1.1.0 |
| dependencies | 1.3.0 |
| inventory_document | 1.1.0 |
| solidity | 1.3.0 |
| source_code | 1.4.0 |
| tls_code | 1.1.0 |
| tls_live | 1.0.0 |
| qsi-engine | 1.3.0 |
| OSV.dev API | v1 |

# Sources and normative documents

Reviewed: 2026-10-13


1. A. Becker, L. Ducas, N. Gama, T. Laarhoven — New directions in nearest neighbor searching with applications to lattice sieving (SODA 2016; IACR ePrint 2015/1128) — IACR ePrint. https://eprint.iacr.org/2015/1128 (peer-reviewed article: classical sieving in 2^{0.292n + o(n)})
2. A. Chailloux, J. Loyer — Lattice sieving via quantum random walks (ASIACRYPT 2021; IACR ePrint 2021/570) — IACR ePrint. https://eprint.iacr.org/2021/570 (peer-reviewed article: 2^{0.2570d + o(d)}, previously 2^{0.2653d + o(d)} (Laarhoven))
3. A. Chailloux, M. Naya-Plasencia, A. Schrottenloher — An Efficient Quantum Collision Search Algorithm and Implications on Symmetric Cryptography (ASIACRYPT 2017; IACR ePrint 2017/847) — IACR ePrint. https://eprint.iacr.org/2017/847 (peer-reviewed article: Õ(2^{2n/5}) time, O(n) qubits, Õ(2^{n/5}) classical memory)
4. C. Chevignard, P.-A. Fouque, A. Schrottenloher — Reducing the Number of Qubits in Quantum Discrete Logarithms on Elliptic Curves (IACR ePrint 2026/280) — IACR ePrint. https://eprint.iacr.org/2026/280 (revision of a EUROCRYPT 2026 paper: 3.12n + o(n) qubits; for n = 256, 1193 qubits and 22 runs of 2^38.98 Toffoli gates)
5. C. Gidney — How to factor 2048 bit RSA integers with less than a million noisy qubits (arXiv:2505.15917) — arXiv. https://arxiv.org/abs/2505.15917 (preprint (2025-05-21): < 1 million noisy qubits, < 1 week, 0.1% gate error, 1 µs surface-code cycle)
6. C. Gidney, M. Ekerå — How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits (Quantum 5, 433) — Quantum. https://doi.org/10.22331/q-2021-04-15-433 (peer-reviewed article (2021): 10^-3 gate error, 1 µs cycle, 10 µs reaction time)
7. C. Zalka — Grover's quantum searching algorithm is optimal (Phys. Rev. A 60, 2746, 1999; arXiv:quant-ph/9711070) — APS. https://doi.org/10.1103/PhysRevA.60.2746 (peer-reviewed article: optimal for any success probability; parallel search gains no more than splitting the space)
8. CRYSTALS-Kyber — Algorithm Specifications and Supporting Documentation (round 3, 2021-08-04) — CRYSTALS team. https://pq-crystals.org/kyber/data/kyber-specification-round3-20210804.pdf (NIST round-3 submission: core-SVP cost 2^{0.292β} classical, 2^{0.265β} quantum; Kyber768 166 quantum bits)
9. Common Vulnerability Scoring System v3.1: Specification Document — FIRST. https://www.first.org/cvss/v3.1/specification-document (final)
10. D. J. Bernstein — Cost analysis of hash collisions: Will quantum computers make SHARCS obsolete? (2009) — cr.yp.to. https://cr.yp.to/hash/collisioncost-20090823.pdf (paper dated 2009-08-23)
11. D. Litinski — How to compute a 256-bit elliptic curve private key with only 50 million Toffoli gates (arXiv:2306.08585) — arXiv. https://arxiv.org/abs/2306.08585 (preprint (2023-06-14): 6000 modules × 1152 physical qubits, one key per 10 minutes under its architecture assumptions)
12. FIPS 203 — Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM) — NIST. https://csrc.nist.gov/pubs/fips/203/final (final (August 2024))
13. FIPS 204 — Module-Lattice-Based Digital Signature Standard (ML-DSA) — NIST. https://csrc.nist.gov/pubs/fips/204/final (final (August 2024))
14. FIPS 205 — Stateless Hash-Based Digital Signature Standard (SLH-DSA) — NIST. https://csrc.nist.gov/pubs/fips/205/final (final (August 2024))
15. G. Brassard, P. Høyer, A. Tapp — Quantum Algorithm for the Collision Problem (LATIN '98; arXiv:quant-ph/9705002) — Springer / arXiv. https://arxiv.org/abs/quant-ph/9705002 (peer-reviewed article: collisions of r-to-one functions in O((N/r)^{1/3}) expected evaluations)
16. J. Proos, C. Zalka — Shor's discrete logarithm quantum algorithm for elliptic curves (Quantum Inf. Comput. 3(4), 2003; arXiv:quant-ph/0301141) — Quantum Information & Computation. https://doi.org/10.26421/QIC3.4-3 (peer-reviewed article)
17. L. K. Grover — A fast quantum mechanical algorithm for database search (STOC 1996) — ACM. https://doi.org/10.1145/237814.237866 (peer-reviewed article)
18. M. Amy et al. — Estimating the cost of generic quantum pre-image attacks on SHA-2 and SHA-3 (SAC 2016; arXiv:1603.09383) — Springer / arXiv. https://arxiv.org/abs/1603.09383 (peer-reviewed article: SHA-256 about 2^153.8 surface-code cycles on 2^12.6 logical qubits; SHA3-256 about 2^146.5 cycles on 2^20)
19. M. Cain et al. — Shor's algorithm is possible with as few as 10,000 reconfigurable atomic qubits (arXiv:2603.28627) — arXiv. https://arxiv.org/abs/2603.28627 (preprint (2026-03-30): P-256 in a few days with 26,000 physical neutral-atom qubits; RSA-2048 one to two orders of magnitude longer)
20. M. Ekerå, J. Gärtner — A high-level comparison of state-of-the-art quantum algorithms for breaking asymmetric cryptography (arXiv:2405.14381) — arXiv. https://arxiv.org/abs/2405.14381 (preprint (2024-05-23): Regev-style algorithms show no overall advantage yet for cryptographic RSA and finite-field sizes)

21. M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018) — IEEE. https://doi.org/10.1109/MSP.2018.3761723 (peer-reviewed article; source of the x + y > z migration inequality)
22. M. Roetteler, M. Naehrig, K. Svore, K. Lauter — Quantum resource estimates for computing elliptic curve discrete logarithms (ASIACRYPT 2017) — IACR / arXiv. https://arxiv.org/abs/1706.06752 (peer-reviewed article: at most 9n + 2⌈log2 n⌉ + 10 qubits and 448 n^3 log2 n + 4090 n^3 Toffoli gates)
23. NIST IR 8547 — Transition to Post-Quantum Cryptography Standards — NIST. https://csrc.nist.gov/pubs/ir/8547/final (final (2026-10-27; the initial public draft of November 2024 is obsoleted). Tables 2 and 4: ECDSA, EdDSA and RSA signatures, finite-field and elliptic-curve DH and MQV, and RSA key establishment are deprecated after 2030 and disallowed after 2035 at every key size — the draft deprecated only the 112-bit level. A composite (hybrid) key establishment stays FIPS 140-validatable while one input comes from an approved scheme. Repeats the federal deadlines of OMB M-26-15: post-quantum key establishment by 2030-12-31 and signatures by 2031-12-31 for high value assets and high impact systems)
24. NIST — Post-Quantum Cryptography Standardization: Evaluation Criteria, Security (Call for Proposals, Section 4.A.5) — NIST. https://csrc.nist.gov/Projects/Post-Quantum-Cryptography/Post-Quantum-Cryptography-Standardization/Evaluation-Criteria/Security-(Evaluation-Criteria) (security categories: AES-128 key search 2^170/MAXDEPTH quantum gates, AES-192 2^233/MAXDEPTH, AES-256 2^298/MAXDEPTH; MAXDEPTH 2^40–2^96)
25. P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997) — SIAM. https://doi.org/10.1137/S0097539795293172 (peer-reviewed article)
26. R. Babbush et al. — Securing Elliptic Curve Cryptocurrencies against Quantum Vulnerabilities: Resource Estimates and Mitigations (arXiv:2603.28846) — arXiv. https://arxiv.org/abs/2603.28846 (preprint (2026-03-30, v2 2026-04-15): 256-bit ECDLP with < 1200 logical qubits and < 90 M Toffoli gates (or < 1450 and < 70 M); < 500,000 physical qubits at 10^-3 error, minutes)
27. S. Jaques, M. Naehrig, M. Roetteler, F. Virdia — Implementing Grover oracles for quantum key search on AES and LowMC (EUROCRYPT 2020; arXiv:1910.01700) — Springer / arXiv. https://arxiv.org/abs/1910.01700 (peer-reviewed article: lower cost estimates for the AES key searches that define the NIST categories)

# Appendices

## A. Cryptographic inventory

| Algorithm | Parameter | Quantum status | Classical status | Count | Locations | Examples |
|---|---|---|---|---|---|---|
| RIPEMD-160 | RIPEMD-160 | weak classically | legacy | 1 | 1 | src/cbmpc/crypto/base_hash.cpp:70 |
| SHA-1 | SHA-1 | weak classically | deprecated | 1 | 1 | src/cbmpc/crypto/base_rsa_oaep.cpp:86 |
| AES-192 | AES-192 | reduced margin (Grover), approved | acceptable | 1 | 1 | src/cbmpc/crypto/base.cpp:126 |
| AES-256 | AES-256 | reduced margin (Grover), approved | acceptable | 1 | 1 | src/cbmpc/crypto/base.cpp:128 |
| HKDF | | reduced margin (Grover), approved | acceptable | 3 | 3 | include-internal/cbmpc/internal/crypto/base_hash.h:351; include-internal/cbmpc/internal/crypto/base_pki.h:57; src/cbmpc/crypto/base_hash.cpp:212 |
| HMAC | | reduced margin (Grover), approved | acceptable | 1 | 1 | src/cbmpc/crypto/base_hash.cpp:146 |
| AES-128 | AES-128 | Grover: acceptable to NIST, below ECCG/ANSSI guidance | acceptable | 1 | 1 | src/cbmpc/crypto/base.cpp:124 |
| BLAKE2 | | Grover: acceptable to NIST, below ECCG/ANSSI guidance | acceptable | 1 | 1 | src/cbmpc/crypto/base_hash.cpp:68 |
| SHA-2 | | Grover: acceptable to NIST, below ECCG/ANSSI guidance | acceptable | 2 | 2 | include-internal/cbmpc/internal/zk/fischlin.h:49; src/cbmpc/crypto/base_hash.cpp:55 |
| SHA-3 | | Grover: acceptable to NIST, below ECCG/ANSSI guidance | acceptable | 1 | 1 | src/cbmpc/crypto/base_hash.cpp:63 |

| | | | | | | |
|---|---|---|---|---|---|---|
| ML-KEM | ML-KEM-768 | post-quantum | acceptable | 2 | 2 | include-internal/cbmpc/internal/crypto/mlkem768.h; src/cbmpc/crypto/mlkem768.cpp:8 |
| BN254 (alt_bn128) | | vulnerable (Shor) | acceptable | 2 | 2 | include-internal/cbmpc/internal/crypto/base_bn256.h; src/cbmpc/crypto/base_bn256.cpp |
| ECDH | | vulnerable (Shor) | acceptable | 5 | 5 | include-internal/cbmpc/internal/protocol/pve_base.h:213; src/cbmpc/api/pve_base_pke.cpp:325; src/cbmpc/api/pve_batch_ac.cpp:308 |
| ECDSA | secp256k1 | vulnerable (Shor) | acceptable | 10 | 10 | include-internal/cbmpc/internal/crypto/base_ecc.h:17; include/cbmpc/api/curve.h:10; src/cbmpc/api/curve_util.h:13 |
| ECDSA | | vulnerable (Shor) | acceptable | 15 | 15 | include-internal/cbmpc/internal/crypto/base_ecc_secp256k1.h; include-internal/cbmpc/internal/protocol/ecdsa_2p.h; include-internal/cbmpc/internal/protocol/ecdsa_mp.h |
| ECDSA | P-256 | vulnerable (Shor) | acceptable | 1 | 1 | include/cbmpc/api/curve.h:9 |
| Ed25519 | | vulnerable (Shor) | acceptable | 7 | 7 | include-internal/cbmpc/internal/crypto/base_eddsa.h; include/cbmpc/api/eddsa_mp.h; include/cbmpc/api/hd_keyset_eddsa_2p.h |
| Ed25519 | Ed25519 | vulnerable (Shor) | acceptable | 12 | 12 | include-internal/cbmpc/internal/protocol/eddsa.h:6; include-internal/cbmpc/internal/protocol/hd_keyset_eddsa_2p.h:26; include/cbmpc/api/eddsa_2p.h:8 |
| ElGamal | | vulnerable (Shor) | deprecated | 4 | 4 | include-internal/cbmpc/internal/crypto/elgamal.h; include-internal/cbmpc/internal/zk/zk_elgamal_com.h; src/cbmpc/crypto/elgamal.cpp |
| Paillier | | vulnerable (Shor) | acceptable | 15 | 15 | include-internal/cbmpc/internal/crypto/base_bn.h:46; include-internal/cbmpc/internal/crypto/base_paillier.h:8; include-internal/cbmpc/internal/protocol/ecdsa_2p.h:18 |
| Pedersen commitment | | vulnerable (Shor) | acceptable | 4 | 4 | include-internal/cbmpc/internal/zk/zk_pedersen.h:8; src/cbmpc/protocol/ecdsa_2p.cpp:28; src/cbmpc/zk/zk_paillier.cpp:537 |
| RSA | | vulnerable (Shor) | acceptable | 3 | 3 | include-internal/cbmpc/internal/crypto/base_rsa.h; src/cbmpc/crypto/base_rsa.cpp:96; src/cbmpc/crypto/base_rsa_oaep.cpp |

| | | | | | |
|---|---|---|---|---|---|
| Schnorr (BIP-340) | vulnerable (Shor) | acceptable | 2 | 2 | include-internal/cbmpc/internal/crypto/base_ecc.h:299; src/cbmpc/crypto/base_ecc.cpp:373 |
| Threshold ECDSA | vulnerable (Shor) | acceptable | 13 | 13 | include-internal/cbmpc/internal/protocol/ecdsa_2p.h:10; include-internal/cbmpc/internal/protocol/ecdsa_mp.h:9; include-internal/cbmpc/internal/protocol/hd_keyset_ecdsa_2p.h:3 |

Long lists are grouped here; the full per-file list is in the JSON report, signed with the same key.

## B. JSON

qsi.report/1 · SHA-256 f97305d07a10199298791bef31824379c96cf9e73e16151592d84131b0a588d9

## C. Input files

| File | Size | SHA-256 | Redaction |
|---|---|---|---|
| include-internal/cbmpc/internal/core/convert.h | 8184 | a56e2e6c773cbb9de2df1dae1b5240c9162c788f2b59d596262f67e417fd9ef5 | 0 |
| include-internal/cbmpc/internal/core/extended_uint.h | 1264 | 8551e894af07214d5d4faf8fd32eb9c6d8fb5da19097bb2f5d5fcb323484aed0 | 0 |
| include-internal/cbmpc/internal/core/log.h | 2878 | 52ec905cb9fa56db485bb9ac1e0d5c82dfb63db54b3b4a7aba1120aba12efac9 | 0 |
| include-internal/cbmpc/internal/core/strext.h | 873 | f245b9bb300b256ecb1b52bc1ae9f5685411a16ab67724cf60bf16bfe17a2053 | 0 |
| include-internal/cbmpc/internal/core/utils.h | 7111 | b98488440036d38243111e9d5126426e71a0481921bf0ae35b099dd3af572413 | 0 |
| include-internal/cbmpc/internal/crypto/base.h | 5757 | def0f4da84c6da78a3ec08c2a1dc2f00f71bf0fb4bf3d788ea9dfff923b7ba01 | 0 |
| include-internal/cbmpc/internal/crypto/base_bn.h | 8887 | 2ab03b1f64160824d980856f8bb4d91a3413b293c5ff6b897e6ddb825bdef696 | 0 |
| include-internal/cbmpc/internal/crypto/base_bn256.h | 4992 | 253d3c7e26e94204dad124a6e6450db90ae1dc3617c30d56c25d300b37bf0ff1 | 0 |
| include-internal/cbmpc/internal/crypto/base_ec_core.h | 18124 | d675354dcb9d373c9e84a80ea4af72476caaa1c708a36ceed0330798a9c0a0e9 | 0 |
| include-internal/cbmpc/internal/crypto/base_ecc.h | 13930 | 4622c8393c4f4bf8d3abdc12789b328ec85e725874e01b02df1896b8c5b311d0 | 0 |
| include-internal/cbmpc/internal/crypto/base_ecc_secp256k1.h | 2905 | 4fd96cb7eca67bfa1627efa01a11f095c5467120c3fca5cda2cce50935884dc2 | 0 |
| include-internal/cbmpc/internal/crypto/base_eddsa.h | 2547 | 11b91f8dd6e32ed727a8838aae477b5198d73a0f46930c8fbf67fdfde33de3ac | 0 |

| | | | |
|---|---|---|---|
| include-internal/cbmpc/internal/crypto/base_hash.h | 9641 | 4eea85e07fd6c13ffbc81910fa739f11f7b4b367607592fdfd203379b14caab0 | 0 |
| include-internal/cbmpc/internal/crypto/base_mod.h | 5488 | 9416b0b19f16e95b1a82029433dc8f2e606cfb21a5e9776f74f1b8b88a6ad2a9 | 0 |
| include-internal/cbmpc/internal/crypto/base_paillier.h | 7140 | 635074e16e9ac9d0941b425940da73aaa1cfcbf9d9bfc23dbcaa17220462cfb1 | 0 |
| include-internal/cbmpc/internal/crypto/base_pki.h | 7471 | 98b0c1f3a7cfed44bcf1f4326ca61c7f25f0f196b63c04c50349ca400cff8a83 | 0 |
| include-internal/cbmpc/internal/crypto/base_rsa.h | 4133 | d57c2cbddf86fcabca1fece5f47377f3c6593e2bbe1fd0d609ce39f666925d18 | 0 |
| include-internal/cbmpc/internal/crypto/commitment.h | 5024 | 12acf4a73daffe317049c4adbf897cac2471d3b5399e8ff8a6338397d3763e8a | 0 |
| include-internal/cbmpc/internal/crypto/ec25519_core.h | 1819 | 91af646c59ed5ed8b7adfb862229c2569f7f7f7e02b06cbfc322ad6de1cd758d | 0 |
| include-internal/cbmpc/internal/crypto/elgamal.h | 3841 | 3e7e1dc52f7c63d54377c18ed4d65cb8e2b7bd01daaf085530e3431ba66b676c | 0 |
| include-internal/cbmpc/internal/crypto/lagrange.h | 2296 | 98372b80b7f725bcc14ed22b542e2e4b0fb959393a8c5e0048af309b2d4d1546 | 0 |
| include-internal/cbmpc/internal/crypto/mlkem768.h | 701 | f34c16ad0e3074f42661f769d8b68b6b72ef35c431db68982049d8f2816620bd | 0 |
| include-internal/cbmpc/internal/crypto/ro.h | 4056 | a2a83967f8151a94001dd3cb20c359a2946abb0bceb359df96cbf243e6467bb6 | 0 |
| include-internal/cbmpc/internal/crypto/scope.h | 1338 | 6ee20f03d2bc5e27c80255120ac9d21650e222971584093b821cb2989ce83669 | 0 |
| include-internal/cbmpc/internal/crypto/secret_sharing.h | 6159 | 183ec4af5557f2176b790d99316b7296041d40dab13c75e712bdd46f4bcb49f8 | 0 |
| include-internal/cbmpc/internal/crypto/tdh2.h | 4119 | 84dcbc72243ec5d7acbc9e44387a9ea28bfe25478d7dfed9b339463b65d0e4a6 | 0 |
| include-internal/cbmpc/internal/protocol/agree_random.h | 1227 | 6b3da20d98680a9f583ec4339f34444ad214ac84e1d96ed194a6bf7a29220153 | 0 |
| include-internal/cbmpc/internal/protocol/committed_broadcast.h | 2828 | 8248d3fd59ab317e917b1a8a7b45500e35e74c587392b028714f95d1912fb872 | 0 |
| include-internal/cbmpc/internal/protocol/ec_dkg.h | 4175 | 47f244abb153a3bcc5efec66471fb70d0b92d13188783d09bf07c823213ce03d | 0 |
| include-internal/cbmpc/internal/protocol/ecdsa_2p.h | 4524 | bb2d45c7091bcb2dbdc81e77b6712c519093577ceec7a937003e57228bd767bb | 0 |
| include-internal/cbmpc/internal/protocol/ecdsa_mp.h | 4370 | 09e920ca00569b73af88dd9ac50b4e809d20e6ef6c57a8ada392a2a40e7ebf3f | 0 |

| | | | |
|---|---|---|---|
| include-internal/cbmpc/internal/protocol/eddsa.h | 802 | 1bae54da49de50c02f780d2bd27bb548eadc22ad94e095c01ae16b7e3cd34270 | 0 |
| include-internal/cbmpc/internal/protocol/hd_keyset_ecdsa_2p.h | 1460 | 98cef577118dc981109e7eca78bf1f41d24967f84449bf597008d2c8a69c604c | 0 |
| include-internal/cbmpc/internal/protocol/hd_keyset_eddsa_2p.h | 1401 | 0ec98767b365b0bcc4a52f1dbf47fb1096c0a580321abe70532ce3c0efacad91 | 0 |
| include-internal/cbmpc/internal/protocol/hd_tree_bip32.h | 1749 | 66bc9c67289298cc3d3bae9f1ba2a960aadcdc50e9069105fad3e1fae2edfcad | 0 |
| include-internal/cbmpc/internal/protocol/int_commitment.h | 847 | d64fd9bd118676bcd8f856a70de1bf1678fe9eeea3597f164ac180e67ca4780d | 0 |
| include-internal/cbmpc/internal/protocol/mpc_job.h | 17919 | a93a6744fe1693909d3e46a1f5976961df525f515c6b28f6c65e4094fbf14763 | 0 |
| include-internal/cbmpc/internal/protocol/ot.h | 8615 | da042352cfa725c622efe54930f3019ff25ee9668e81cc0586b68c687e67a1e1 | 0 |
| include-internal/cbmpc/internal/protocol/pve.h | 1268 | f3d48fad161594f0175ece90e99aa661df33257447229f54ae10ec09f1bf046f | 0 |
| include-internal/cbmpc/internal/protocol/pve_ac.h | 4481 | e6d60929413bf4fa70c899856329d5f096062b0f39fc3d0e74e92c58f94d9824 | 0 |
| include-internal/cbmpc/internal/protocol/pve_base.h | 9109 | 26dcc28fef1f2733ed7c10f2f9a389df73ecd8cc981358367c67b29d930ebfc1 | 0 |
| include-internal/cbmpc/internal/protocol/pve_batch.h | 2350 | 140959a47e0bb81fb502192fc4d3c56a4f69b4aae8cd1b642ca8a5dcd6789575 | 0 |
| include-internal/cbmpc/internal/protocol/schnorr_2p.h | 955 | 26b5a68de67cb25384741b84bf862f3c30ab0f505474cc0b0a5f8fc10b8eb71b | 0 |
| include-internal/cbmpc/internal/protocol/schnorr_mp.h | 1636 | 25c8b2c1a79f70349ee2098eb56c8bb6f148598fccc91b214af185a17b0b0c1b | 0 |
| include-internal/cbmpc/internal/protocol/sid.h | 2398 | a13269fefc215f1f5dbddd924174ee1fed95db295d1cdc909c69c5c014bbd589 | 0 |
| include-internal/cbmpc/internal/protocol/util.h | 2083 | 21d08c60af4b75538821a8ec31f3cd8a50adc6c8753c206837f9f7bb558ea170 | 0 |
| include-internal/cbmpc/internal/zk/fischlin.h | 3695 | 792546bb45cf6ce43db2bd34723b18da117d1f635a97fe518d09ff02c8fb31b6 | 0 |
| include-internal/cbmpc/internal/zk/small_primes.h | 1046 | 2159754496de246e69d96310f1f9dacaec9df1f3c04c16385c4bce995ea2e83e | 0 |
| include-internal/cbmpc/internal/zk/zk_ec.h | 2816 | 01853d5739fecf852b82e20193aad9faa1a4d38702e4e6dc202d54094b3728e3 | 0 |
| include-internal/cbmpc/internal/zk/zk_elgamal_com.h | 3006 | 90fc7c4c62e170e4fc2bf0dad335cbf556472bb70f30ae1d39ef37dbb55a5135 | 0 |

| | | | |
|---|---|---|---|
| include-internal/cbmpc/internal/zk/zk_paillier.h | 7613 | 5ad37919ff30625baf38930da6bea9aefe1fb3cddab2a02e52d9e42a6a0f2253 | 0 |
| include-internal/cbmpc/internal/zk/zk_pedersen.h | 4139 | 899fad68ffd0cc058ea9edabd9505a26a539d2d31e64f3345d976991216bc528 | 0 |
| include-internal/cbmpc/internal/zk/zk_unknown_order.h | 622 | f632c986fe305b9253cd61b00bf6ca4e91ae2afd4228d910f7ac3219feb9e1ef | 0 |
| include-internal/cbmpc/internal/zk/zk_util.h | 1721 | 5751b4ae4aa64f5fbe7af8802b7a489737f5f042d05873dcee21ca8fec55a69e | 0 |
| include/cbmpc/api/curve.h | 305 | 2facde4582730feb655e92033395aada9402fe2c32f78e6ac475bd8324936cad | 0 |
| include/cbmpc/api/ecdsa_2p.h | 3871 | 105375859252d5a634822b7af6726a4a80003591cc2c8bcefdb6bf72674a1542 | 0 |
| include/cbmpc/api/ecdsa_mp.h | 6405 | ab364b34f5c08cdcc436596ba385938b4853fb25bbc630ef507be7fa21d46588 | 0 |
| include/cbmpc/api/eddsa_2p.h | 3468 | 16ec1fa4dcb7e197b2ec1edb4f5fee3330b9d4b5016221c0292ca5c9208445d3 | 0 |
| include/cbmpc/api/eddsa_mp.h | 6602 | 3ab97cfccc9ad4c9ed383358f267e68140e69926bbafa9d98fa10ea2c09d477c | 0 |
| include/cbmpc/api/hd_keyset_ecdsa_2p.h | 1761 | fc16145b6c7db1ebf924e06172dbc041aca2a8b7186a79755d7f2cc88fb36399 | 0 |
| include/cbmpc/api/hd_keyset_eddsa_2p.h | 1946 | ae8c48d856ea67d1aed7044aa7ab0df1e682a6a0fd062f8c8114fcbc75ff8070 | 0 |
| include/cbmpc/api/pve_base_pke.h | 7256 | 9280cee649b4752fc92360b22e6ba7ba8e6a2f588cb3bba2a9280a4135f23c2e | 0 |
| include/cbmpc/api/pve_batch_ac.h | 6780 | 3fbe4705e6454184aec00705496ef78bcea0307f975fad7d1d77f0493e57cf8a | 0 |
| include/cbmpc/api/pve_batch_single_recipient.h | 3445 | f98c566cdba6caf7b3d6da68af9a773ae7946dc8f11451af43d573610a54ba57 | 0 |
| include/cbmpc/api/schnorr_2p.h | 3929 | 5b151a5677a5496d763f4e8f678a78b6b570dd8196d16f7eecd30605da3c5dc1 | 0 |
| include/cbmpc/api/schnorr_mp.h | 7143 | 39b31ba9980a43fb1fc8ba8a220ef25492b147ae020b1d0e540fe42b2d6bb53b | 0 |
| include/cbmpc/api/tdh2.h | 4229 | 4292de2890a66de4ec19bd642ebbb68c0653bfd9e97957dc1d0153dee033308a | 0 |
| include/cbmpc/c_api/access_structure.h | 2766 | e9d53711590e479dd37c19a6c4d82cd08c6e9c1438121edb270c0690a9e71b9e | 0 |
| include/cbmpc/c_api/cmem.h | 1198 | 29624c2da9423a7572f131eb244e216c2fe7944dfe59319edb3458b733c56fb4 | 0 |
| include/cbmpc/c_api/common.h | 2499 | 94bc94a3e819a719afa35151029c8964fb157cd7eb2609d93f257cd975b86746 | 0 |
| include/cbmpc/c_api/ecdsa_2p.h | 4212 | 88ab538b235f8a78490328bc6bf2ce39657689eab758413d36fdf7b00841f476 | 0 |
| include/cbmpc/c_api/ecdsa_mp.h | 8348 | 08b76cba5186c15ccfc1c7b6e49c290a4d4ab7c0fda98f3fab66eb3271a9bbed | 0 |

| | | | |
|---|---|---|---|
| include/cbmpc/c_api/eddsa_2p.h | 3582 | 751f5fe0b4c9bdf3245a51887076846a0e89b0d6fc0ccbebc64c8470c92114f9 | 0 |
| include/cbmpc/c_api/eddsa_mp.h | 8026 | 22e2811912998b1feb41b8ddb171695f639e00af18c8258ebebe5b1e7de48bb9 | 0 |
| include/cbmpc/c_api/job.h | 2853 | b61893a3c68d4ff836d4ffc6ca21bfa2601cc1b8c5161389217505ccaeee8d16 | 0 |
| include/cbmpc/c_api/pve_base_pke.h | 9594 | 7a93b2954d8f05745ba82a3238fe482f2c87aa593b60bb3f07048151c557b80d | 0 |
| include/cbmpc/c_api/pve_batch_ac.h | 5993 | 2468fbb957d0607ea714fe4a5aecb91f8d46871053485c81f9d2490637dffda5 | 0 |
| include/cbmpc/c_api/pve_batch_single_recipient.h | 3786 | f5d0f75b826da34508313415d8ce966191e701ce3184b230ee5cb0a3c8a84f43 | 0 |
| include/cbmpc/c_api/schnorr_2p.h | 3659 | 078fc3e11670dae87921ecc42238204196314aca571f4b328a726d2ea725b9e4 | 0 |
| include/cbmpc/c_api/schnorr_mp.h | 8242 | daa14190b82a6adbe781120039be8f5f5d873c55e50d583c26569512d8b06dd6 | 0 |
| include/cbmpc/c_api/tdh2.h | 5404 | 67aac4a7c9032fa87988d1fdc559ae3b18df5773c4fcd0b59303afb6e0464cf1 | 0 |
| include/cbmpc/core/access_structure.h | 2181 | 168f318f2676eca879dd6567bc22aad1390c9f0217ce916134ec04a1b8e2a318 | 0 |
| include/cbmpc/core/access_structure_limits.h | 109 | f6eb33c9c3f3ab6886e672f6b71ae35e21fd9895433c5d0845ea6036e66f4567 | 0 |
| include/cbmpc/core/bip32_path.h | 383 | f404682cee2ab3b94e54ab3a625d74c6c483779aed72389cd63407ad0d3bd954 | 0 |
| include/cbmpc/core/buf.h | 8626 | 3211ba3cefa7bb2dc584fc4854d146b9830afcaa7040b64f31e4de89875d8c06 | 0 |
| include/cbmpc/core/buf128.h | 3122 | 8e2bef19a7dcab978acb3c29a992e12072f764b53f449d97e2dc907d5027c35c | 0 |
| include/cbmpc/core/buf256.h | 1780 | 5c99e90645842aa88fa75191c7e138fe458b4605ae4b06b112e125aed7ba69e2 | 0 |
| include/cbmpc/core/error.h | 2750 | 82b6b7a47256c7fab6f46e74313f97291a634811f28cb64dd1486d95628182da | 0 |
| include/cbmpc/core/job.h | 2073 | 0c35a15cdae4d5f4450d93e5ec27af30cc5550bf9722da8f2d41016fd71fc85f | 0 |
| include/cbmpc/core/macros.h | 946 | da42b82206cd7ae447f0dbb023494ba8cc4a94f36977faccd8a82e2648d5c50b | 0 |
| include/cbmpc/core/precompiled.h | 3021 | 45071365b97849cfe36fea65e1a63b25a09078189e33495006cc592f8c44561a | 0 |
| src/cbmpc/api/access_structure_util.h | 9006 | ceddfd95a56fef01a49c9f9be5d4d02f9fa6f94dc584bf1e911b0289fc1f4580 | 0 |
| src/cbmpc/api/curve_util.h | 1106 | 2b1ce8031d294bd7721265d8bd13469b0828e5511b10a43ef2ba9cb16f3dd4d6 | 0 |
| src/cbmpc/api/ecdsa2pc.cpp | 13715 | 2bd6f2c9780b68b323f33f000c019d99aa9ece77fc399fd69a4c418a5b247724 | 0 |
| src/cbmpc/api/ecdsa_mp.cpp | 23511 | e8b790be6ff65035680333e944c16aa409fc39b9f3efde09f3e1a034b66fb069 | 0 |
| src/cbmpc/api/eddsa2pc.cpp | 9662 | 277a7491b190d4e1fbec7aac06aa60e357a4a72d374a5af0d9c8d323344ab2ad | 0 |
| src/cbmpc/api/eddsa_mp.cpp | 23718 | fdc84660705d7dd8168aca64fd9620da0a9f5bc0ae584beded77339b79502e0b | 0 |

| | | | |
|---|---|---|---|
| src/cbmpc/api/hd_keyset_ecdsa_2p.cpp | 9399 | 46f38a66df82ed9cd573f6887e08a8dd6f32f96cd4fa3d9e0baaa8efa2c9b34a | 0 |
| src/cbmpc/api/hd_keyset_eddsa_2p.cpp | 8196 | db69a2795c6ad2b92dccdb42301e4d58701cfe694787171382580d1f6eac8a5c | 0 |
| src/cbmpc/api/hd_keyset_util.h | 1102 | 80b15e70053540d811ad516f0a2c9d20577454365379a737793359870c44884e | 0 |
| src/cbmpc/api/job_util.h | 2963 | 0eaf11ceb45fe2a8c36148f925434e2b9b4e55b053f16fac7c023f751d8928c2 | 0 |
| src/cbmpc/api/mem_util.h | 2091 | fd6fb01ab217ab64ab6182fe01ee9f656381a6690848ab3f0b11c6506b7063d0 | 0 |
| src/cbmpc/api/pve_base_pke.cpp | 15533 | 5639e943156dbf5e142c178b07da2c4eb0a5d60bcfa0f61af1b18bc22b830937 | 0 |
| src/cbmpc/api/pve_batch_ac.cpp | 19720 | 8b8be49db0d9dabae40d79f31809585672d745b3ac0a66147f6bab96a2f7daa3 | 0 |
| src/cbmpc/api/pve_batch_single_recipient.cpp | 12533 | c5572970684fdeff67948f2d3e40b6e2777a4fbe0737e70eb334ecaf7cc0e746 | 0 |
| src/cbmpc/api/pve_internal.h | 6257 | 47bce2585e9a7bf7b3a1939dc6f914f9f4a71a8e745a1dd372bc00ebab713754 | 0 |
| src/cbmpc/api/schnorr2pc.cpp | 10080 | 9495eab05f3110eaff559f33a89cea322a17fd6c873139300a9ff54b042eaaad | 0 |
| src/cbmpc/api/schnorr_mp.cpp | 24257 | bd909c04140c085cb023ee5fac58ee26b361234b6ce65f395fc0f37defe39bd0 | 0 |
| src/cbmpc/api/tdh2.cpp | 15729 | a4ff1563e8b6429c4576c0430a6164510211b943d461362ee0ebd3a7186ae1c8 | 0 |
| src/cbmpc/c_api/access_structure_adapter.h | 4841 | 08d2877d3dd792e33afd088e767d5925d177be052c50f2023524c0bbf81ece68 | 0 |
| src/cbmpc/c_api/common.cpp | 1592 | 5280698eda4f9f4e66b6ae8560818f4c8dd2f4f9f7c13ce2957f583ffc70640f | 0 |
| src/cbmpc/c_api/ecdsa2pc.cpp | 8935 | 350195de0a3b6dec093519c569d2fb7bb50de40ec80f81f750e6fe2e5cef5a77 | 0 |
| src/cbmpc/c_api/ecdsa_mp.cpp | 14657 | ae5651ed5d7de9eea96f9e9b1c9b8f6ce4779bde23df3c7ad5835c584fe57986 | 0 |
| src/cbmpc/c_api/eddsa2pc.cpp | 7405 | cac76cf191e1737f82300959bafe1b6d9a73723da0ac752c4fe573764ab576ad | 0 |
| src/cbmpc/c_api/eddsa_mp.cpp | 14249 | 7393ff3f085a404140f443da26bae530000d8158963983a77a4eebe9402e9849 | 0 |
| src/cbmpc/c_api/pve_base_pke.cpp | 19582 | c6e5ec7ce741d2c32c9895618ec807b02782e502401b759a79a7cd1030c1ba7e | 0 |
| src/cbmpc/c_api/pve_batch_ac.cpp | 15057 | 86de96b4539a390203c7c25dd9f452ee31f6d22efe2445135d19fbfb356f9640 | 0 |
| src/cbmpc/c_api/pve_batch_single_recipient.cpp | 9558 | c1bf050dfe843b0c9491835b2a4a67d5a87bc31e91b20167c76e18fccb15a40b | 0 |
| src/cbmpc/c_api/pve_internal.h | 3922 | 67717ecc213db88a2ceeefeca34434d6ba44e7606d169118a384f4d0c70ae884 | 0 |
| src/cbmpc/c_api/schnorr2pc.cpp | 8125 | 0ddf4832e2ea31c50a5933a5f0d4b82c7fbbaa50e3de6f4b79f58ec4d8dfe6fc | 0 |

| | | | |
|---|---|---|---|
| src/cbmpc/c_api/schnorr_mp.cpp | 15011 | f1c40a8f09a77764fe725b393cdc7c0b4e5af68ec1373471d09a0e43b6105cb4 | 0 |
| src/cbmpc/c_api/tdh2.cpp | 12864 | 8a7813233276b5b2322215aff73847238307574f781aa9a331e8e619538f38ee | 0 |
| src/cbmpc/c_api/transport_adapter.h | 3637 | b40563601e342dad20c6d714eeb50b71e2a1e17fb3c55ea6e1b06781dd91d199 | 0 |
| src/cbmpc/c_api/util.h | 6137 | 54e197b005d5bf85a970ffd48c13fa55c1748b8e1a53e8b70cd4d2194e73d6a3 | 0 |
| src/cbmpc/core/buf.cpp | 19249 | 4f05b7b7f064a8df4fdbe8f73930690073f9bdf7fe644ad8df2fc95974436711 | 0 |
| src/cbmpc/core/buf128.cpp | 8097 | ab4b91df60b609f7958c7c9927b1bfac57fef68079ae644be0070555c944d7d6 | 0 |
| src/cbmpc/core/buf256.cpp | 5718 | bc92ef88c227313eb5c334e8315985ca4ddc23c0cd543e7bcf3778aa9d688878 | 0 |
| src/cbmpc/core/convert.cpp | 5831 | e058331387e5a1467cbc044ad88e4717644b4893d4d251e1268798a4d8c75f05 | 0 |
| src/cbmpc/core/error.cpp | 5599 | b32bfc0d066f34f00f5b5e319a04284d35b405ace58c487e0f2105293e024aeb | 0 |
| src/cbmpc/core/strext.cpp | 3904 | d760ebfff69eb90c43825f4c67cd57c619e447baa7219ee06a3f1f675b5874bb | 0 |
| src/cbmpc/crypto/base.cpp | 8596 | dd2af36fb426bc7b3b618d9972a4215e01577766256eac1ed92966b54e57af93 | 0 |
| src/cbmpc/crypto/base_bn.cpp | 21760 | fbec856dfe302bc6c8dc4dfdaa52a0f0ee84287ef13634885cf34e1d03ade10e | 0 |
| src/cbmpc/crypto/base_bn256.cpp | 38165 | 5aab4b8a2ae1328950c40a1589bee877ecb9468101b0fc7baecd773514e5fec3 | 0 |
| src/cbmpc/crypto/base_ec_core.cpp | 2539 | 16280192dd3f73e0969ff983499669d5327b215018dace8f4e37eb161e822c6a | 0 |
| src/cbmpc/crypto/base_ecc.cpp | 35151 | dd31645addc8cf00b027fc453868a4aec33a826a95a7acca172dd3af54ca97ec | 0 |
| src/cbmpc/crypto/base_ecc_secp256k1.cpp | 15422 | 31cabe4ca85327316c07f9865c9efa3a81c98a630bc3276806d3dd96cf400c73 | 0 |
| src/cbmpc/crypto/base_eddsa.cpp | 8170 | 45697966a1eb28d114a3f26c52dbd3d97abef3ed0bd13a6da051f2825f70f99d | 0 |
| src/cbmpc/crypto/base_hash.cpp | 12718 | 6797b8c75aaaf50070644024ab4df41ade4e6a676dc53cba207929a5e334af86 | 0 |
| src/cbmpc/crypto/base_mod.cpp | 16690 | 8390bd8433403219ac67b744ca8c1f04b919376f75b48e12aaebe1044a551738 | 0 |
| src/cbmpc/crypto/base_paillier.cpp | 11598 | bc74a8ec5b010ffa69aaeabeb2edb7dd075972229f2997cb48b6bc38106de467 | 0 |
| src/cbmpc/crypto/base_rsa.cpp | 13948 | ce1f7e2af4ab105d95931ebce00de933dc0718afbb555bea8cac329324b39cf4 | 0 |
| src/cbmpc/crypto/base_rsa_oaep.cpp | 7631 | 488a05f19bfe3be1bb45df021c887b46f88af1930dbe88f4412830a77a70cd19 | 0 |
| src/cbmpc/crypto/drbg.cpp | 2236 | 4e3fbc5382e267539b72d1556890ad8d3ffb15509db8ba4f6d3609f4a75b7708 | 0 |
| src/cbmpc/crypto/ec25519_core.cpp | 32300 | fcf18843c539fff85668d46a094aa97b4ba6014fdb3005bbf08ce87804fa4d61 | 0 |
| src/cbmpc/crypto/elgamal.cpp | 4096 | 44e90812713ef86a0f2ef69a55d8fd53447a6aee718fe91410a73a3e27b0ab16 | 0 |

| | | | |
|---|---|---|---|
| src/cbmpc/crypto/lagrange.cpp | 6291 | 4ed9ad1c1a9f9a3597cd49fc2034d9b6c0986b8d92fb7b1109a87f2dda24772c | 0 |
| src/cbmpc/crypto/mlkem768.cpp | 3026 | 4078550dd5d763189b297626094dc68bca380049f1f36a8b123a0a0272576792 | 0 |
| src/cbmpc/crypto/ro.cpp | 4612 | b594d9dff0634336ee222e03972a16e871028b334c15cbd03b74de7bcc74bb41 | 0 |
| src/cbmpc/crypto/secret_sharing.cpp | 18838 | aea152036abdf368422312bc67b533c9564db6645b423f87bcd7bb4a0fbcce26 | 0 |
| src/cbmpc/crypto/tdh2.cpp | 7393 | 09bb2ea74cb90fce2455dfdfb59a46d2602ec8ec22a88cc65f1006746eaa87db | 0 |
| src/cbmpc/protocol/agree_random.cpp | 4095 | fda790e3f41acc7561402f8dc85f21c20e40d9a0228ee0e8596c2135e1476175 | 0 |
| src/cbmpc/protocol/ec_dkg.cpp | 24991 | 3b23c2ec0047bc420818c27c59c8c134214d805ed9f9e835757aa7b5a07a1855 | 0 |
| src/cbmpc/protocol/ecdsa_2p.cpp | 24824 | 676da782f28e8371b19dd2737d91d08ac8fbf805bf4c91b0c4defe0f9116216c | 0 |
| src/cbmpc/protocol/ecdsa_mp.cpp | 20263 | 6fd98e15fd272931e7ed76418a3833da33c3facd895e60d95453dd3acea59c0b | 0 |
| src/cbmpc/protocol/eddsa.cpp | 982 | e534d848e59ef7fad0a8156804d0b545e919aca37cc0995bee5482904f5e985e | 0 |
| src/cbmpc/protocol/hd_keyset_ecdsa_2p.cpp | 6599 | dc2f999c8ba5817ac88817a08aa3cdfbfb09d258bf16c2b11719990f9355b075 | 0 |
| src/cbmpc/protocol/hd_keyset_eddsa_2p.cpp | 5751 | 60b1f34c8280d8ca1979f5522553544df9015d0c6c3f8d0f46c9a4f411d1f5b7 | 0 |
| src/cbmpc/protocol/hd_tree_bip32.cpp | 1202 | b477c98d2d5e834999eab5bc3bdb6d610d01137b3922ce40d2b33f296cfcd939 | 0 |
| src/cbmpc/protocol/int_commitment.cpp | 90896 | 28356c850e19c96fc65a8abb43a8191779f756cf424c07146d4ffde86e64afdd | 0 |
| src/cbmpc/protocol/mpc_job.cpp | 1207 | 50346b5c5839ece7d5db342babcff1a124f3341a79ee0f4cc9f5da526af5fb09 | 0 |
| src/cbmpc/protocol/ot.cpp | 17534 | 6a1a3e82170609e3e7d4bf15b7d0fcdb22c2887d3f68b851ad9aabe7bb0800db | 0 |
| src/cbmpc/protocol/pve.cpp | 4987 | d4c53ffa9710407b9d016673f2de6eb5ce18e3458a6c803b11eb57880905286f | 0 |
| src/cbmpc/protocol/pve_ac.cpp | 11348 | de010bed8070f48c7adafbaf606cc0e4bb43243177c9c12448ecb3d9dc2ca536 | 0 |
| src/cbmpc/protocol/pve_base.cpp | 1661 | 4cf97abd43be32195eeb17a2e8ef41aec4bf40d7dac414bee458425764c25df3 | 0 |
| src/cbmpc/protocol/pve_batch.cpp | 7350 | 2bbf74000b73ade87272a6d048a4c894019a5338da4916084e26f510be5dd178 | 0 |
| src/cbmpc/protocol/schnorr_2p.cpp | 5348 | a998fa463bf108155247586360a6e309dc8ec940ea10916c93a66334c0baab50 | 0 |
| src/cbmpc/protocol/schnorr_mp.cpp | 7312 | b7aa492e64186b649ce0a19abc70e27afe5f7bcaaffc008bad3a6e0ee7d83af8 | 0 |
| src/cbmpc/zk/fischlin.cpp | 2071 | b272d3a5cc641eeaab74baaf6d494958a0e56db0e205d28f301b9944b3d66c68 | 0 |

| | | | |
|---|---|---|---|
| src/cbmpc/zk/small_primes.cpp | 82305 | 508021bcc27b6bc7efe6481ce1f85407a02501c9a8a6c1628cd022fc59ec4630 | 0 |
| src/cbmpc/zk/zk_ec.cpp | 11090 | 864cc7b1616050ed50842d23b2c8d78c09ca253fcd811d08987091f4bf002614 | 0 |
| src/cbmpc/zk/zk_elgamal_com.cpp | 11105 | ba7214e14d37fcb23c9334f0502c9ce9fb9627cce992fd676921d82c6f8377ec | 0 |
| src/cbmpc/zk/zk_paillier.cpp | 23160 | fd1dc137a76118088153562685210 63e54f833048b8fc42e616a59c7188e0ab0 | 0 |
| src/cbmpc/zk/zk_pedersen.cpp | 18743 | 968641640c623fc2cdfe6012b8686db394631006f90478bfc182820228662bc6 | 0 |
| src/cbmpc/zk/zk_unknown_order.cpp | 2674 | bcde5bc6688e9e3f6bc022f2a4089d98f98a8d3b83f277ed00fbde0e1608fbd7 | 0 |